Project Charter

Smart+: Monitoring Student Well-being

Prince Sultan University

SE499 | Software Engineering Design and Development

CS499 | Computer Science Senior Project

Supervised by:

Dr. Tariq Soussan

Dr. Khalid Tarmissi

Prepared by:

Naif Aba Alrous, 222110926

Bader Alforij, 221110864

Mohammed Nour Aldin, 221110436

Ziyad bin Juwayid, 221110548

Abdullah Babkeer, 221211369

Saud Alqadhib, 220211257

Table of Contents

List of Tables

List of Figures

Project Description

This project watches over student well-being during class using Apple Watch readings. A simple iPhone app connects to the watch and sends health readings to our system. The system checks for unusual signs and raises alerts when needed. Online views support the work for campus doctors to review cases. When a doctor accepts a case, the student gets a clear message with the next steps.

Core readings for the first version include heart rate, heart rhythm variation, blood oxygen, breathing rate, and wrist temperature. Blood pressure appears only when the watch or a paired device provides it.

Project Scope

The project delivers a classroom health support system from setup to pilot. It covers collecting selected Apple Watch readings during class, storing them safely, and turning them into clear alerts that staff can act on. It includes a simple class view for campus doctors with accept or reject actions, matching students to suitable doctors, and notifying students after a decision. The scope also includes role-based sign-in, consent records, basic activity logs, quick screen updates, and a focused pilot on one course section with a demo and brief training.

Project Objectives

The primary objectives of the Classroom Health Monitoring System are as follows:

1. **In-class monitoring:** Capture key Apple Watch readings during scheduled class hours for early awareness.
2. **Actionable alerts:** Detect unusual patterns with clear rules and generate understandable alerts.
3. **Role based views:** Provide a simple status and a triage queue for campus doctors.

4. **Doctor routing and response:** Match alerts to suitable on duty doctors and record accept or reject decisions.
5. **Student notification:** Notify students promptly when a doctor accepts and share next steps.
6. **Privacy, security, and reliability:** Enforce consent and minimal data sharing, secure handling, audit logs, quick updates, and high availability during class hours.

Main Stakeholders

Primary stakeholders

- **Students**

  Wear the watch, give consent, receive notifications, follow guidance.

- **Campus Doctors/Clinicians**

  Review triage queue, accept or reject cases, and provide care instructions.

- **Health Center Administration**

  Manage doctor schedules, specialties, and service hours; oversee triage policies.

Supporting stakeholders

- **University IT**

  Maintain servers, dashboards, user accounts, and network access.

- **Data Privacy and Compliance Officer**

  Ensure consent, data use, and retention meet policy and law.

- **University Administration**

  Approve pilot scope, allocate budget, and set success criteria.

- **Accessibility and Student Affairs**

Advise on inclusive use, student support, and communications.

- **Security Team**

  Review risk, access control, and incident response plans

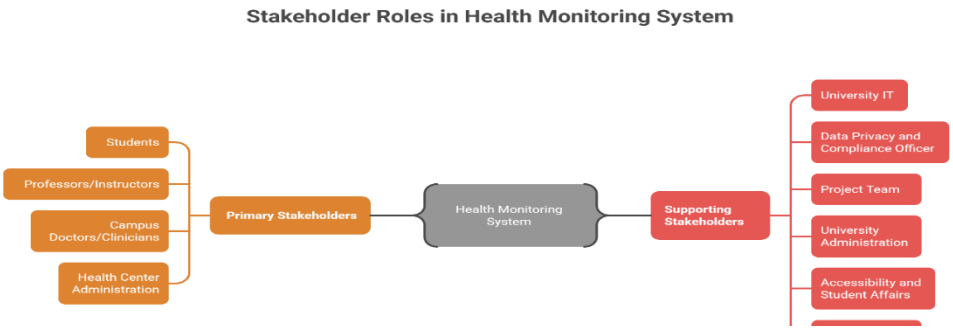


Figure 1 : Stakeholders

Risk

This risk register lists the key risks that could affect the delivery, safety, or adoption of the system. Each risk includes a short event description, impact, likelihood, an overall rate, and clear mitigation steps. An owner is assigned to every risk, so actions are tracked and reviewed during weekly checks. The register will be updated as the pilot progresses and new information becomes available.

Table 1: Risk

Risk	Risk Event	Impact	Likelihood	Rate	Mitigation	Owner
Health data unavailable	A watch or a phone does not provide readings	High	Medium	High	Prompt to re-enable permissions, show status, retry from cache	iOS Lead
Permissions denied	Student declines health access	Medium	Medium	Medium	Clear consent flow, explain purpose, allow opt-in later	Product Lead

Battery drain	Data collection reduces device battery	Medium	Medium	Medium	Batch reads, background delivery, adaptive sampling	iOS Lead
False alerts	Normal activity triggers alerts	High	Medium	High	Tune rules, short smoothing window, doctor review before student notice	Data/Rules Lead
Missed events	Real issue not flagged	High	Low	Medium	Redundant checks, monitoring, alert audits, clear escalation	Data/Rules Lead
Privacy breach	Unauthorized access to student data	Very High	Low	High	RBAC, encryption, audits, least-privilege access	Security Lead
Compliance breach	PDPL/GDPR or campus policy violation	Very High	Low	High	DPO review, consent records, retention rules, DPIA	Compliance Officer
Doctor unavailable	No suitable on-duty doctor to accept	High	Medium	High	Backup routing, escalation policy, clear student guidance	Health Center Admin
Storage/DB outage	MongoDB unavailable or data loss	Very High	Low	High	Managed cluster, replicas, backups, restore drills	DevOps
Schedule slippage	Key milestones slip	High	Medium	High	Fix scope, track critical path, buffer, early demos	Project Manager

Constraints

The project has technical and non-technical limits that shape scope and delivery.

Technical Constraints

- iPhone only with Apple Watch data; readings depend on Health app permissions
- Background delivery on iOS is limited; real time is best effort
- Campus or mobile network gaps can delay uploads and notices
- Push notifications require working device tokens and APNs setup
- Data quality varies by watch model and how the watch is worn
- Blood pressure appears only if a paired cuff app writes it to Health

Non-Technical Constraints

- One academic semester and a small team with defined roles
- Student consent is required and may limit participation
- Doctor availability varies by schedule, specialty, and gender preference
- Pilot scope limited to a single course section and set class hours
- Training time for clinic staff is limited

Key Performance Indicators (KPIs)

Speed

- Data received in the system within 2 minutes for 95% of readings
- Dashboards refresh within 3 seconds for 95% of alerts
- Student notified within 10 seconds after doctor acceptance for 95% of cases

Accuracy and signal quality

- Valid reading coverage for enrolled students ≥ 90% of class minutes
- False alert rate ≤ 10% per week
- Missed critical alerts: 0 during the pilot

Adoption and engagement

- Student consent rate ≥ 70% in the pilot section
- Doctor median accept time ≤ 5 minutes

Reliability

- System uptime ≥ 99% during class hours
- Notification delivery success ≥ 99.5%

Privacy and compliance

- Reportable privacy incidents: 0
- Access events recorded in audit logs: 100%

Business Needs

The university needs a simple way to spot student health issues during class and route them to the right campus doctor without exposing private details to teaching staff. The clinic needs a triage queue that shortens time to care and records accept or reject decisions. The pilot should provide evidence on safety, response times, and adoption to guide scale-up, budgeting, and policy alignment.

Value Created

- Improves in-class safety and reduces unnoticed events
- Lowers classroom disruption with minimal, privacy-safe status views
- Gives the clinic measurable response times and clearer records for follow-up

Sustainability Model

- University funding tied to student safety and well-being programs
- Grants or partnerships for health innovation pilots and equipment
- Low ongoing costs through focused scope, shared dashboards, and basic maintenance routines

What Sets It Apart

This system focuses on real classroom safety with fast, private alerts. Campus doctors get a triage queue with accept or reject actions and guided routing to the right doctor based on specialty, availability, and gender preference. The loop closes by notifying the student after a doctor accepts. The design is small, focused, and ready for a class pilot.

Key points that make it unique

- Classroom first use case with short, clear alerts and minimal disruption
- Doctor in the loop acceptance so students get real guidance, not auto messages
- Smart routing that respects specialty and gender preferences
- Simple path to pilot: one class section, quick setup, measurable results

Target Audience

Primary users

- Students who wear an Apple Watch and consent to share readings during class
- Campus doctors and clinic staff who review alerts and provide guidance

Secondary users

- Health center administration that manages schedules and policies
- University IT, privacy, and security teams that support the pilot

Comparative Analysis of Similar Applications

Table 2: Comparative Analysis of Similar Applications

Feature	Fitness apps	Campus safety apps	Telehealth remote monitoring	Our system
Target group	Individuals training	General student body	Clinical patients	Students in class, campus clinic
Core function	Personal goals and trends	Panic alerts, location beacons	Doctor prescribed home monitoring	In class alerts with doctor acceptance
Data source	Wearables for personal use	User reports, location	Medical devices, clinical setup	Apple Watch readings with consent
Privacy model	User controls data in app	Broad alerts to security	Clinical data in hospital systems	Status only to clinical details to doctors
Routing to doctor	Not applicable	Not applicable	Through clinical team	Auto routing by specialty and availability
Classroom fit	Low	Medium	Low	High
Setup effort	User only	App install and contacts	Clinician onboarding	Pilot ready for one class section

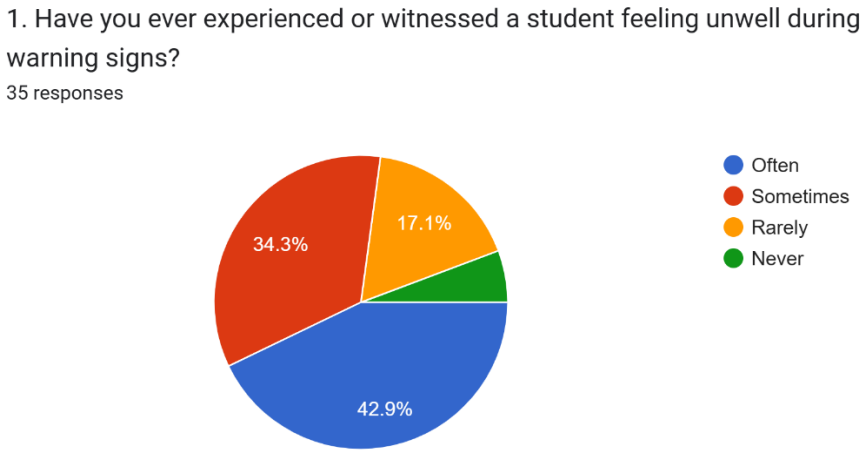
Development Methodology

- **Short stages:** plan in two-week iterations with clear goals and a working demo at the end of each stage.
- **Backlog focus:** keep a small list of must-have items and deliver them first.
- **Regular checks:** brief daily standups, weekly planning, and stage reviews with stakeholders.
- **User feedback:** test each increment with one pilot class and the clinic, then adjust.
- **Quality gates:** definition of done includes working build, basic tests, privacy checks, and a demo that runs end-to-end.
- **Risk control:** track a short risk register, remove blockers early, and keep the pilot scope tight.

Survey Analysis:

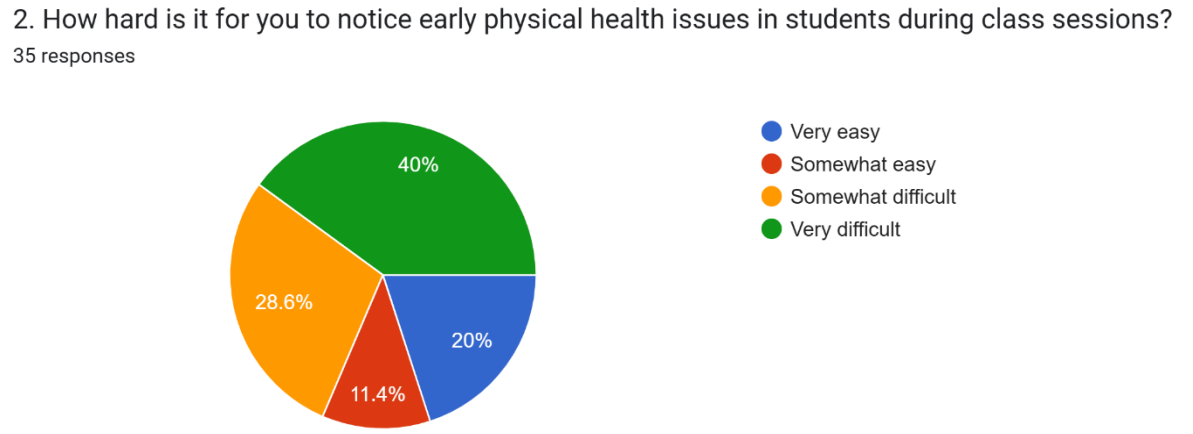
We designed a survey to evaluate current challenges in detecting and responding to student health issues during class time. It focuses on understanding how often health concerns go unnoticed, what delays occur in getting support, and how effective existing methods are in ensuring student safety. The responses help validate the need for real-time classroom health monitoring solution and provide clear insight into the features, alerts, and workflow improvements required to support quicker, safer, and more reliable intervention by medical staff.

Question 1 asked participants whether they had experienced or witnessed a student feeling unwell during class without clear warning signs. Most respondents reported this occurred **often at 42.9%** or **sometimes at 34.3**%, while **17.1% said it happens rarely** and only a small percentage said it never happens. This shows that unexpected health incidents in classrooms are common, reinforcing the need for a system that can detect early signs and provide timely alerts before conditions worsen.

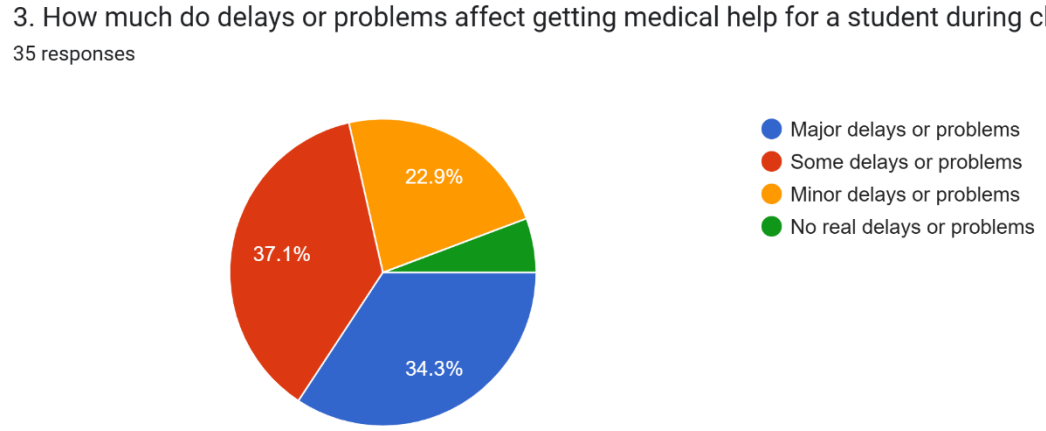


Question 2 asked participants how hard it is to notice early physical health issues in students during class sessions. A large portion found it **very difficult at 40%** or **somewhat difficult at 28.6%,** while only **20% found it very easy** and **11.4% somewhat easy**. This indicates that most

respondents struggle to identify early health problems, highlighting a clear gap that supports the need for real time monitoring and automated health alerts within the Smart+ system.

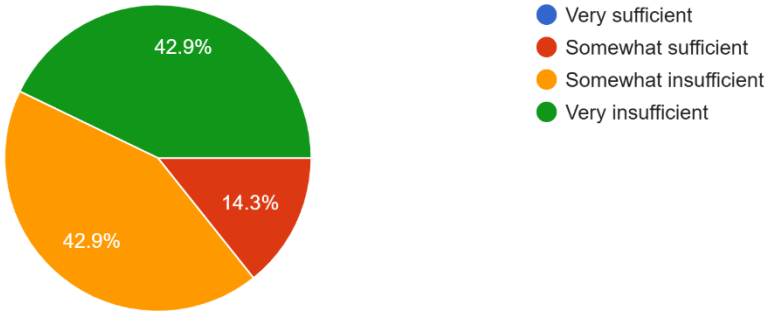


Question 3 asked how much delays or problems affect getting medical help for a student during class. Most respondents reported experiencing either **some delays or problems at 37.1%** or **major delays or problems at 34.3%**, while **22.9% experienced minor delays** and only a small percentage reported no real delays. This indicates that timely access to medical support in classroom situations is often hindered, highlighting the need for a faster and more structured response system.



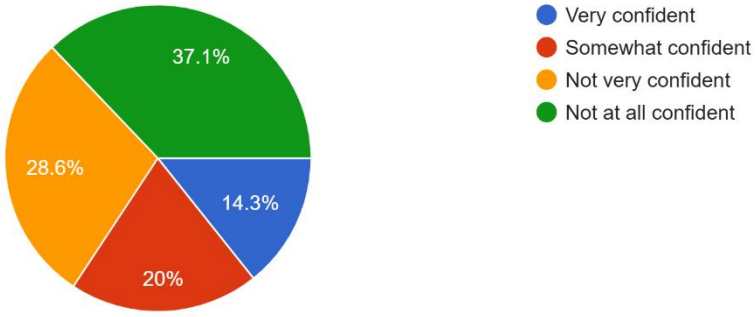
Question 4 asked whether current methods are enough to detect health problems early during lectures. The majority felt these methods were **very insufficient at 42.9%** or **somewhat insufficient at 42.9%**, while only **14.3% considered them somewhat sufficient,** and none viewed them as very sufficient. This clearly shows dissatisfaction with existing detection approaches and supports the need for a smarter, technology-driven monitoring solution.

4. Do you feel current methods are enough to detect health problems early during lectures?
35 responses

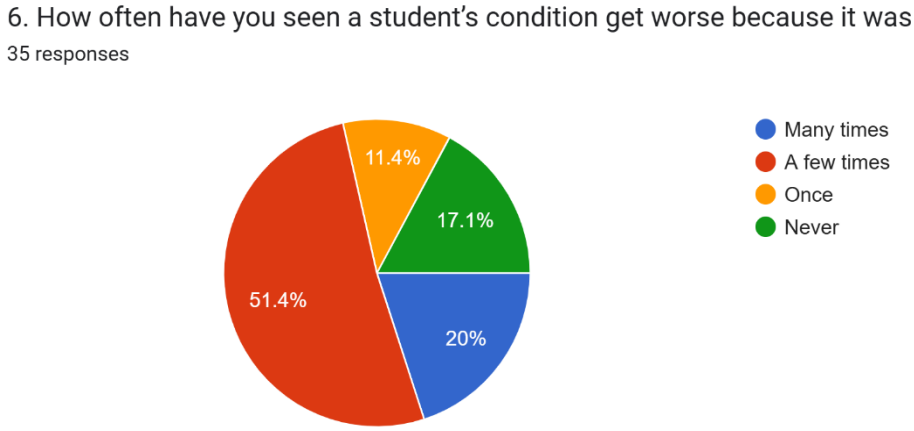


Question 5 asked how confident participants are that serious health issues can be identified on time in the classroom. Most respondents expressed low confidence, with **37.1% not at all confident** and **28.6% not very confident**, while **20% were somewhat confident** and only **14.3% very confident**. This demonstrates a significant trust gap in the current ability to handle serious health situations promptly.

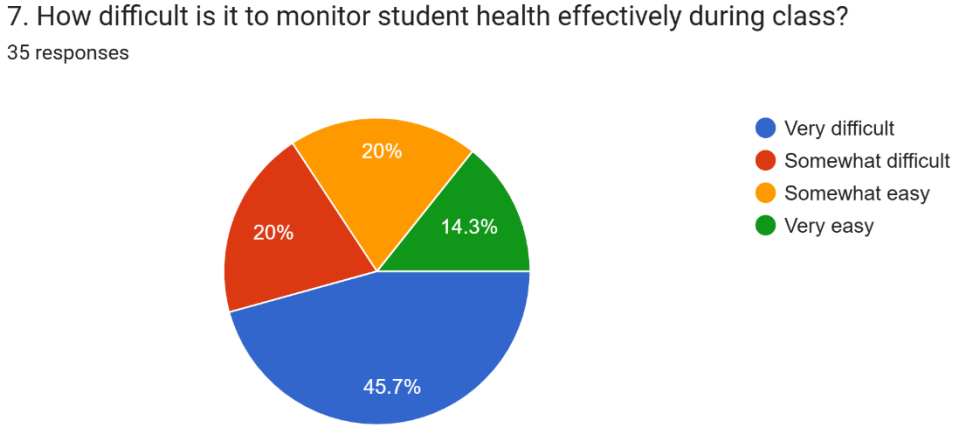
5. How confident are you that serious health issues can be identified on time in the classroom?
35 responses



Question 6 asked how often participants have seen a student's condition worsen because it was noticed too late. More than half reported this happening **a few times at 51.4%**, while **20% said many times** and **11.4% once**. Only **17.1% said it never happened**. This shows that delayed recognition is a frequent issue, reinforcing the need for early detection and alert mechanisms.



Question 7 asked how difficult it is to monitor student health effectively during class. Nearly half found it **very difficult at 45.7%** and **20% somewhat difficult**, while **20% found it somewhat easy** and only **14.3% very easy**. This indicates that effective monitoring is a challenge for most educators, further justifying the implementation of automated monitoring tools.

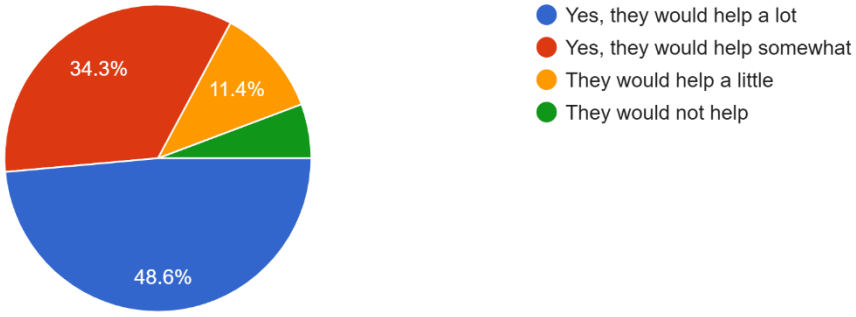


Question 8 asked whether real time health alerts from student devices would help respond faster to health issues in class. An overwhelming majority agreed, with **48.6% stating they would help a lot** and **34.3% saying they would help somewhat**. Only a small portion felt they would help a

little or not at all. This shows strong support for real time alert functionality within the Smart+ system.

8. Would real time health alerts from student devices help you respond faster to health issues in class?

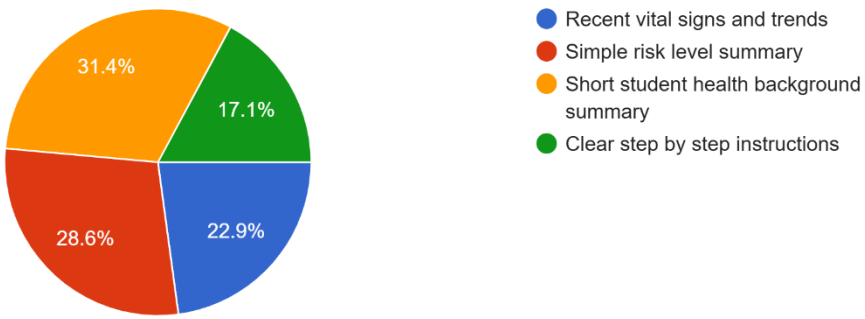
35 responses



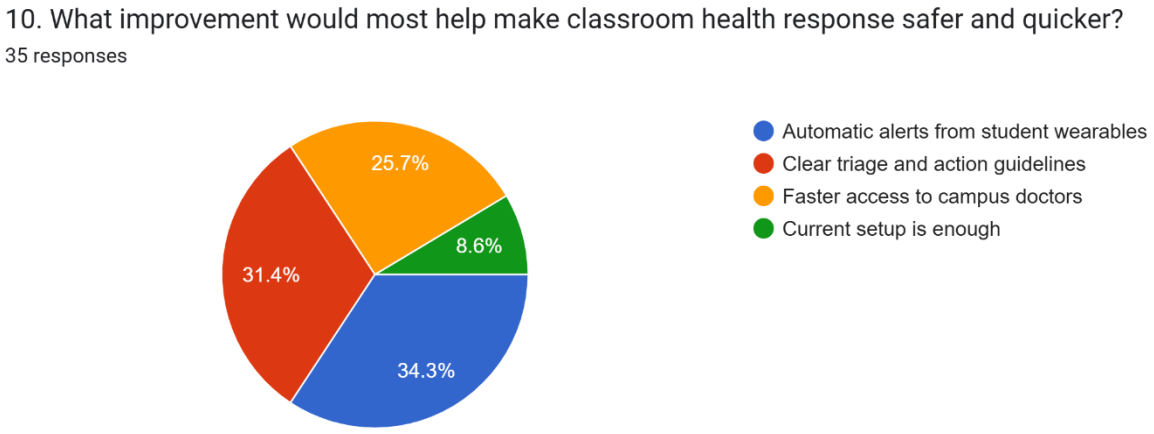
Question 9 asked which type of information would be most helpful when responding to a student health concern in class. The most selected option was a short **student health background summary at 31.4%**, followed by **a simple risk level summary at 28.6%**, **recent vital signs and trends at 22.9%**, and **clear step-by-step instructions at 17.1%**. This suggests that concise and clear contextual information is preferred to support quicker decision-making.

9. Which type of information would be most helpful when you respond to a student health concern in class?

35 responses



Question 10 asked what improvement would most help make classroom health response safer and quicker. The majority selected automatic alerts from **student wearables at 34.3%** and clear triage and **action guidelines at 31.4%**, followed by faster access to **campus doctors at 25.7%**. Only **8.6% believed the current setup is enough**. This highlights a strong demand for system enhancements, particularly automated alerts and structured response workflows, aligning directly with the project goals.



Requirements

Functional Requirements

System Access and Roles

- **FR-1.** The system provides a role based web portal where doctors sign in with email and password and are routed to their dashboard on successful authentication.
- **FR-2.** The system enforces role based access control so only authenticated doctors can view alerts, metrics, and pilot reports.

Permissions and Data Capture (iOS App and Backend)

- **FR-3.** The iOS app requests read permissions for agreed Apple Health signals including heart rate, HRV, SpO_2, respiratory rate, and wrist temperature.
- **FR-4.** The app collects only health signals that the student has explicitly approved in the iOS Health permissions dialog.
- **FR-5.** During scheduled class sessions, the app captures new readings from Apple Watch via HealthKit observers and anchored reads.
- **FR-6.** The app supports delta sync so that only samples created after the last successful anchor are uploaded in each batch.
- **FR-7.** When the phone is offline, the app queues unsent samples locally and retries upload when network connectivity is restored.
- **FR-8.** The app prevents duplicate records after restarts by using anchors and idempotent upload behavior.

Transport, Validation, and Storage

- **FR-9.** The app sends batched JSON payloads containing health readings to a secure ingestion endpoint over HTTPS using an authentication token.
- **FR-10**. The backend API validates incoming requests for schema, timestamps, units, and student identifier and rejects invalid payloads with a clear error response.
- **FR-11**. On successful validation, the backend writes readings to MongoDB using time series collections indexed by student and timestamp.

- **FR-12**. The backend maintains connection pooling and error handling for stable access to MongoDB.
- **FR-13**. Operators can confirm data flow through API logs and by querying MongoDB collections for a student and time range.

Alert Engine and Analytics

- **FR-14.** The alert engine evaluates stored readings using configured thresholds for heart rate, HRV, SpO_2, temperature, and respiratory rate.
- **FR-15**. The alert engine evaluates trends over multiple samples to avoid triggering alerts from a single transient spike.
- **FR-16.** Alerts are generated only when sustained abnormal readings are detected for a student.
- **FR-17**. Each alert is assigned a severity level based on configured rules for moderate and critical abnormalities.
- **FR-18.** The alert engine attaches reasoning metadata to every alert describing the key values and patterns that led to the trigger.

Doctor Queue and Alert Review Workflow

- **FR-19.** The doctor portal displays a list of routed alerts including student identifier, severity, timestamp, and current status.
- **FR-20**. Doctors can open an alert to view detailed vital signs trends, graphs, and reasoning metadata for the alert.
- **FR-21.** The portal shows an initial alert status of Pending when an alert is first created and routed.
- **FR-22.** Doctors can accept or reject a routed alert using dedicated actions on the alert detail screen.
- **FR-23**. On acceptance, the system updates the alert status to Accepted and records the doctor decision in a structured log with timestamp, doctor ID, case ID, and reasoning.
- **FR-24**. On rejection, the system updates the alert status to Rejected and records the rejection reason in the decision log.
- **FR-25.** Doctors can edit and update the rejection reason for an alert while keeping a valid status history.

- **FR-26.** The system enforces valid status transitions for alerts and prevents skipping or entering invalid states.
- **FR-27.** Once an alert is accepted, the portal hides Accept and Reject buttons to avoid duplicate decisions.

Student Registration and Directory

- **FR-28.** The system provides a student registration workflow that records student details and associates them with a device identifier.
- **FR-29**. Saved student information is used by the alert engine and directory service for routing and display.
- **FR-30.** The directory service maintains doctor accounts, specialties, and availability used for routing alerts to suitable on duty doctors.

Email Notification and Guidance

- **FR-31.** For every Accepted alert, the backend initiates a messaging chain to notify the affected student.
- **FR-32.** The system uses Gemini template based generation to produce email guidance text that follows predefined tone and clarity rules.
- **FR-33.** The backend sends an email to the student for each accepted alert and logs the attempt and outcome.
- **FR-34.** The email service retries failed deliveries according to configured retry logic and logs each failure and retry attempt.
- **FR-35.** Acceptance events are not considered complete until a student notification has been attempted.

Monitoring, Metrics, and KPIs UI

- **FR-36.** The backend records performance metrics such as alert evaluation throughput, email latency, and failure counts for later analysis.
- **FR-37.** The system raises backend alerts when message generation or email delivery is slow or failing.

- **FR-38**. The doctor portal provides an Active Cases or KPI dashboard that summarizes active alerts, trends, and key metrics.
- **FR-39.** Doctors can filter alerts by date, metric type, and severity on the dashboard.
- **FR-40**. The dashboard supports sorting and enlarging graphs for closer inspection of trends.
- **FR-41.** The alerts list in the portal refreshes automatically on a fixed cycle so doctors see near real time updates without manual reload.
- **FR-42**. Doctors can export alert or case data for further analysis in external tools.
- **FR-43.** A pilot readiness page displays KPI summaries such as alert accuracy, latency, and notification success for simulated and real sessions.

Workflow Simulation and Pilot Readiness

- **FR-44.** The system supports workflow simulations that generate synthetic class sessions and alert scenarios for readiness testing.
- **FR-45.** The system computes KPIs from simulation results including latency, coverage, accuracy, uptime, and notification success.
- **FR-46.** A readiness checklist aggregates KPI results, load testing outputs, and documentation status to determine pilot readiness.
- **FR-47.** Deployment management supports pilot deployment with post deployment health checks and rollback procedures.

Non-Functional Requirements

Performance and Responsiveness

- **NFR-1.** For 95 percent of readings, data is received and stored in the system within two minutes of capture during class hours.
- **NFR-2.** For 95 percent of alerts, the doctor dashboard or queue view refreshes to show new alerts within three seconds.
- **NFR-3.** For 95 percent of accepted alerts, the student receives a notification email within ten seconds of doctor acceptance.
- **NFR-4.** The doctor portal alert list and main KPI screens load in under three seconds under normal pilot load.

- **NFR-5.** The system maintains stable performance during simulated class load with concurrent alert evaluations and email bursts.

Accuracy, Reliability, and Availability

- **NFR-6.** Valid reading coverage for enrolled students is at least 90 percent of class minutes during the pilot.
- **NFR-7.** The weekly false alert rate remains at or below 10 percent of generated alerts.
- **NFR-8.** The system targets zero missed critical alerts during the pilot through redundancy and rule tuning.
- **NFR-9.** System uptime during scheduled class hours is at least 99 percent.
- **NFR-10**. Notification delivery success is at least 99.5 percent for messages sent to students.

Privacy, Security, and Compliance

- **NFR-11**. The system enforces least privilege access through role based access control for all doctor and operator accounts.
- **NFR-12**. Health data in transit between the iOS app and backend uses HTTPS with authenticated requests.
- **NFR-13**. Health data at rest in MongoDB is stored in a secured managed cluster with replicas and regular backups.
- **NFR-14.** Access to student data is logged in audit logs so that 100 percent of access events are recorded for review.
- **NFR-15.** The system maintains consent records and applies data retention rules aligned with PDPL, GDPR, and campus policy.
- **NFR-16.** The system is designed to avoid exposing detailed health information to teaching staff and limits visibility to authorized clinic staff.

Platform and Operational Constraints

- **NFR-17.** The initial mobile client supports iPhone devices paired with Apple Watch and relies on HealthKit permissions for data access.
- **NFR-18.** Background delivery for readings operates on a best effort basis within iOS limits and queues data when network connectivity is unavailable.

- **NFR-19.** Push notifications depend on valid APNs and device tokens and must degrade gracefully when tokens are invalid or connectivity is missing.
- **NFR-20.** The MongoDB deployment uses replicas and backup and restore drills to protect against data loss.
- **NFR-21.** The project scope limits the pilot to a single course section and defined class hours, and all monitoring and readiness checks are aligned with this scope.

User Stories

User Story Name	User Story Description	Priority
Doctor Login	As a doctor, I want to sign in using my email and password so that I can access the Smart+ dashboard securely.	1
View Alerts Queue	As a doctor, I want to view all incoming health alerts so that I can monitor students at risk.	1
Review Alert Details	As a doctor, I want to open an alert to view vital signs and trends so that I can understand the student's condition.	1
Accept Alert	As a doctor, I want to accept an alert so that the system can notify the student with medical guidance.	1
Reject Alert	As a doctor, I want to reject an alert so that false or non critical cases are filtered out.	1
Update Rejection Reason	As a doctor, I want to edit rejection notes so that the decision history remains accurate.	2
Monitor KPI Dashboard	As a doctor, I want to view performance metrics so that I can track alert activity and system effectiveness.	2
Filter Alerts	As a doctor, I want to filter alerts by severity and date so that I can focus on critical cases.	2
Student Health Data Sync	As a student, I want my Apple Watch data synced automatically so that abnormal health patterns can be detected.	1
Provide Health Permissions	As a student, I want to allow HealthKit access so that my vital signs can be monitored safely.	1
Receive Alert Email	As a student, I want to receive an email when an alert is accepted so that I am informed about my health status.	1

View Pilot Readiness Report	As an operator, I want to view readiness metrics so that I can evaluate system performance before deployment.	2
Simulate Workflow	As an operator, I want to run simulation scenarios so that I can test the alert process reliability.	2
Export Alert Data	As a doctor, I want to export alert data so that I can use it for external analysis and reporting.	3
Automatic Alert Generation	As the system, I want to generate alerts based on abnormal vitals so that doctors are notified in real time.	1
Queue Failed Data Sync	As the system, I want to store unsent data locally so that no health readings are lost offline.	2
Log Decision History	As the system, I want to log doctor decisions so that every action is traceable.	1
Display Real Time Updates	As a doctor, I want the dashboard to refresh automatically so that I see updated alerts instantly.	1

Project Timeline

This timeline illustrates the Smart+ project schedule from the start of development to final pilot readiness and presentation. The project is structured across multiple phases, beginning with an initiation and planning period where requirements, system architecture, and data flow design were defined. This is followed by iterative sprints focused on core implementation, including HealthKit data integration, backend ingestion, alert engine development, and doctor portal construction.

Each sprint includes dedicated stages for development, integration, testing, and refinement of key features such as alert generation, doctor workflows, email notifications, and KPI dashboards. Continuous testing and simulation are carried out to validate performance, accuracy, and system stability before moving to the next phase. The timeline concludes with pilot readiness preparation, final system validation, documentation completion, and project presentation, ensuring the system is fully functional and aligned with real classroom health monitoring requirements.

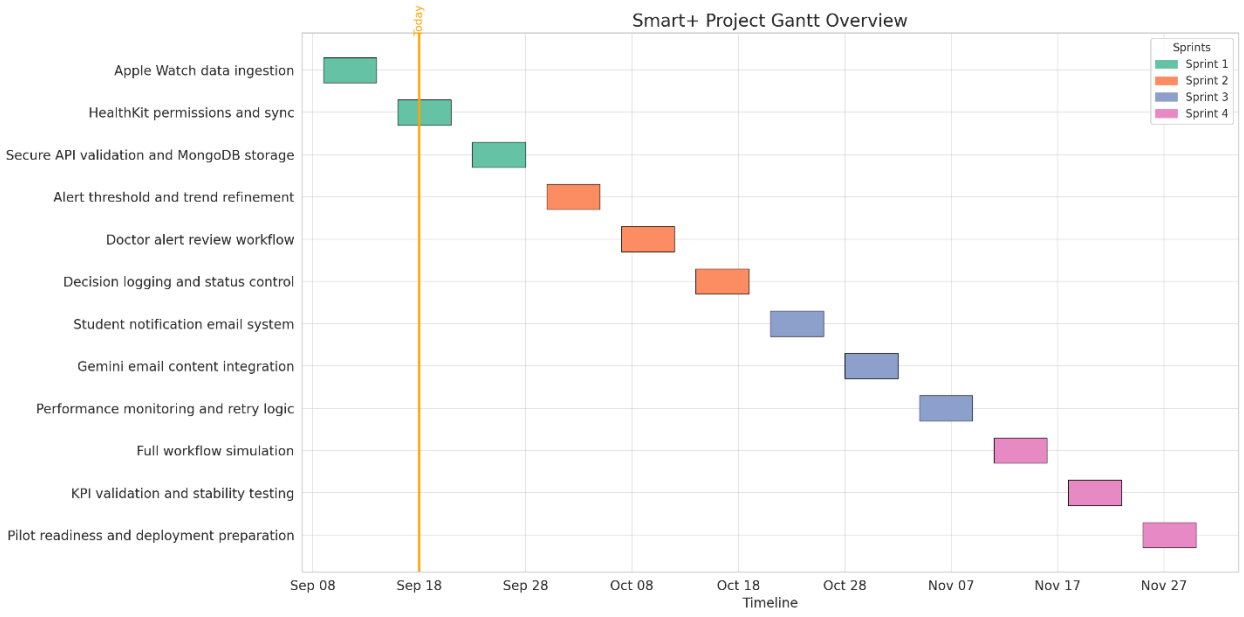


Figure 2: Smart+ sprint timeline with current progress highlighted

System Architecture Overview

This architecture outlines a student health monitoring system. The **Apple Watch** collects vital signs, relayed via the **iOS App** and **HealthKit** to a secure **Backend API**. Data is stored in **MongoDB**, where an **Alert Engine** processes it using rules to identify potential health concerns. Real-time updates and push notifications are managed through **WebSockets/FCM/APNs** and routed by a **Directory Service** to **Doctor Dashboards**. The system prioritizes robust **Privacy & Security** at every stage.

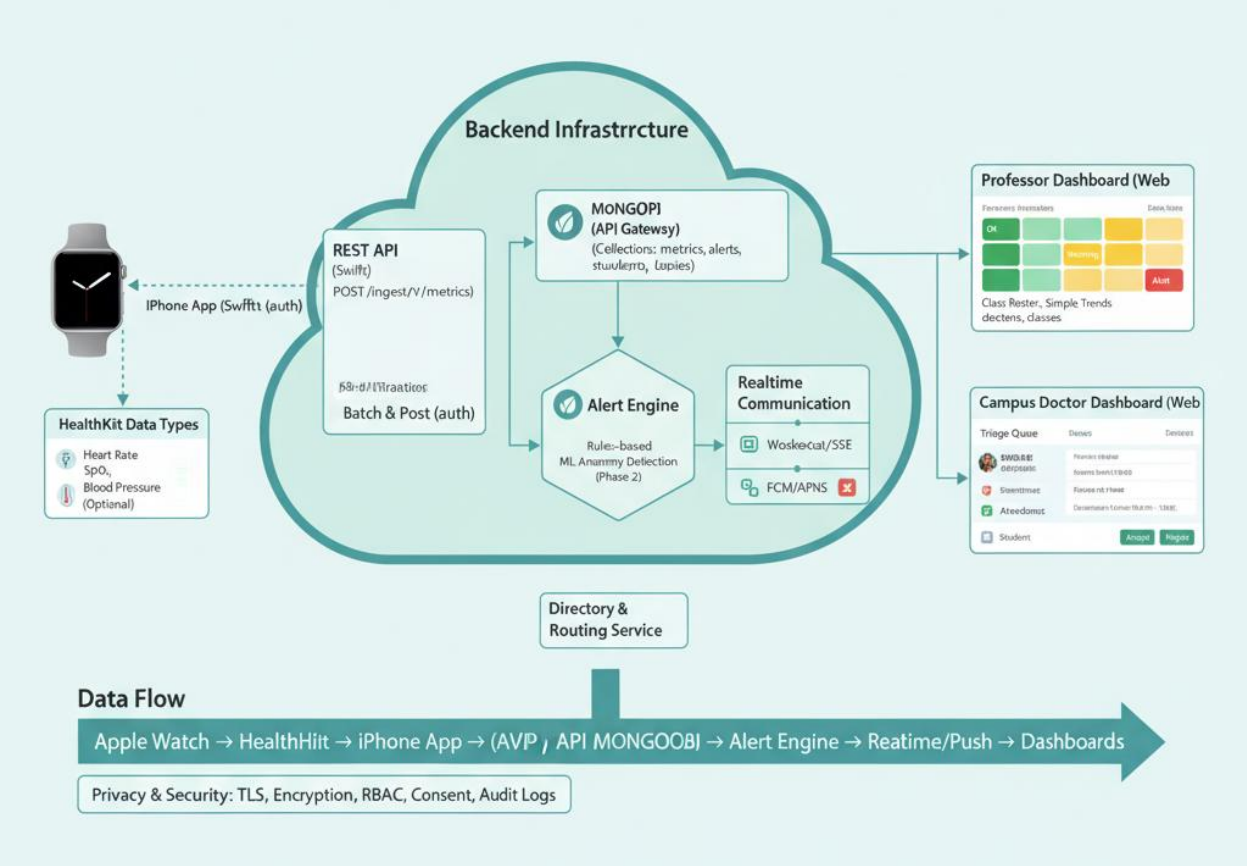


Figure 2: System Architecture Overview

Sprint-01 Overview

A working ingestion pipeline is live. The iPhone app collects approved HealthKit readings and sends them to the backend. The API validates and writes records to MongoDB. Data from a real device is visible in the database.

Sprint -01 Goal

Prove the full data path from Apple Watch to iPhone app to API to MongoDB. Keep it simple, reliable, and easy to verify so we can use it as the base for alerts in the next sprint.

User Stories Implemented

- **As a student**, I can install the app, grant permissions, and have my health readings sent automatically while class is in session.
- **As the system**, I only read data that the student has approved and I send new records without duplicates.
- **As the backend**, I accept valid payloads, reject invalid ones with clear errors, and store records reliably in MongoDB.
- **As an operator**, I can verify that data is flowing by checking API logs and MongoDB collections.

Tasks Implemented

- iOS app setup in Xcode and bundle signing for device install.
- Health permissions screen and request flow for selected signals.
- Background delivery using HealthKit observers and anchored reads.
- Local queue and retry for intermittent network.
- JSON payload schema and batching for uploads.
- Secure API endpoint for ingestion, request validation, and auth token check.

- MongoDB database creation, collections, and indexes for time-series writes.
- Connection pooling and error handling between API and MongoDB.
- Basic server logs and request metrics.
- Deployment of API to server and environment configuration.

Functional Requirements

1. **Permissions and data capture**
    - The app requests read permissions for the agreed health signals.
    - Only approved signals are collected.
2. **Delta sync**
    - The app sends only new samples since the last successful anchor.
3. **Resilience**
    - If the phone is offline, samples queue locally and send when online.
    - Duplicate records are not created after app restarts.
4. **Transport**
    - App sends batched JSON to the ingestion endpoint over HTTPS with an auth token.
5. **Validation**
    - API validates schema, timestamps, units, and student identifier; rejects bad payloads with a clear error.
6. **Storage**
    - Valid records are written to MongoDB with proper indexing for student and timestamp.
7. **Observability**
    - Operators can confirm successful writes via logs and see records in MongoDB.

Unit Test Cases

Test Case: API builds successfully

Table 3: API builds successfully

Test Case ID	UTC-API-01
Requirement	FR-API-01
Preconditions	Source code available; build pipeline or local toolchain ready.
Steps	1) Trigger API build. 2) Observe build output.
Expected Result	Build completes with no errors and produces an artifact.
Actual Result	
Pass/Fail	Pass

Test Case: API deployed on server

Table 4: API deployed on server

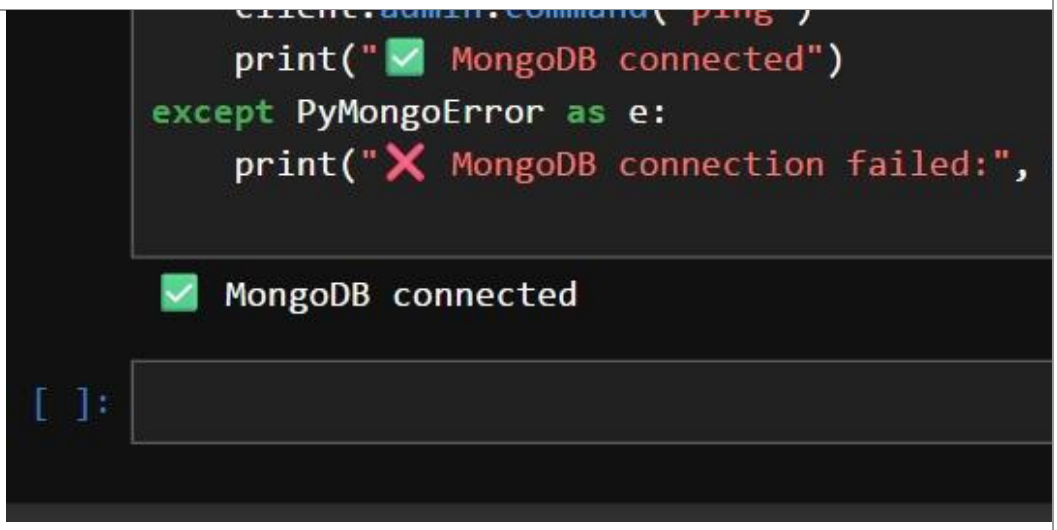
Test Case ID	UTC-API-02
Requirement	FR-API-02
Preconditions	Server reachable; environment variables configured.
Steps	1) Deploy API. 2) Call health/version endpoint.
Expected Result	Health endpoint returns OK with version.

Actual Result	Pretty-print ☑
	{
	"status": "ok",
	"message": "Health Data API running"
	}

Pass/Fail	Pass

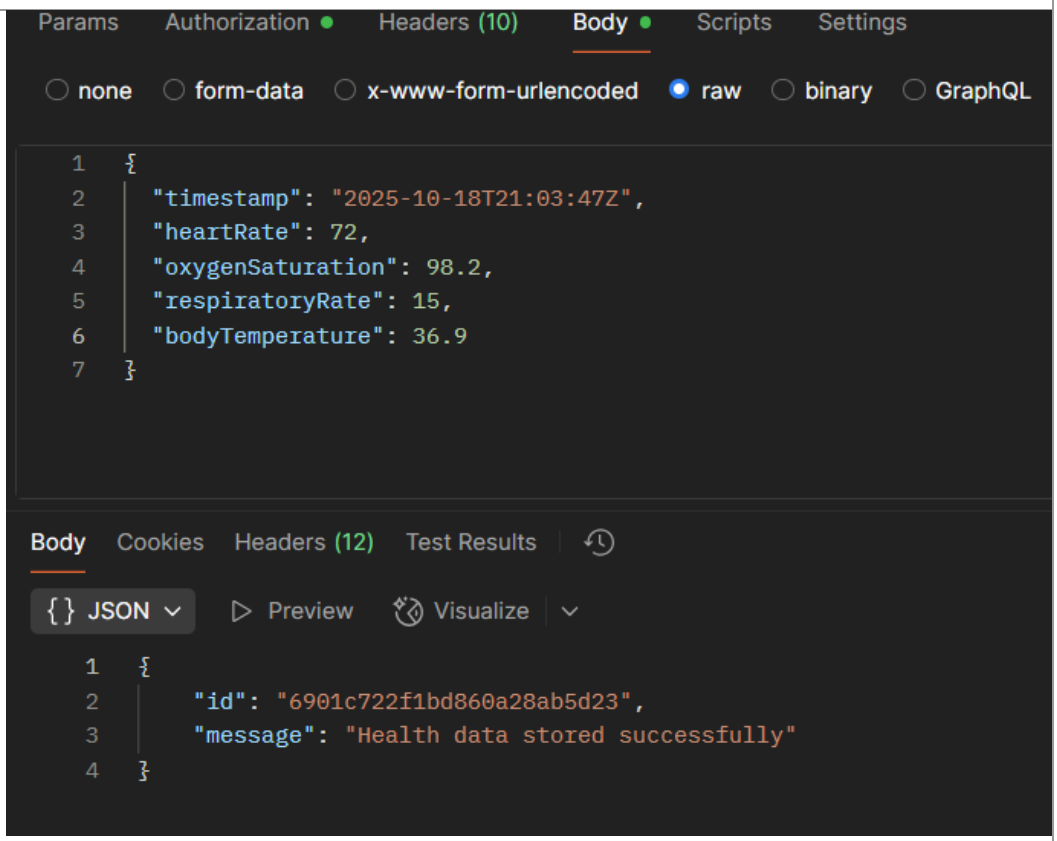
Test Case: API connects to MongoDB

Table 5: API connects to MongoDB

Test Case ID	UTC-DB-01
Requirement	FR-DB-01
Preconditions	MongoDB running; URI and creds set.
Steps	1) Start API. 2) Check startup logs.
Expected Result	Successful DB connection logged; no connection errors.
Actual Result	
Pass/Fail	Pass

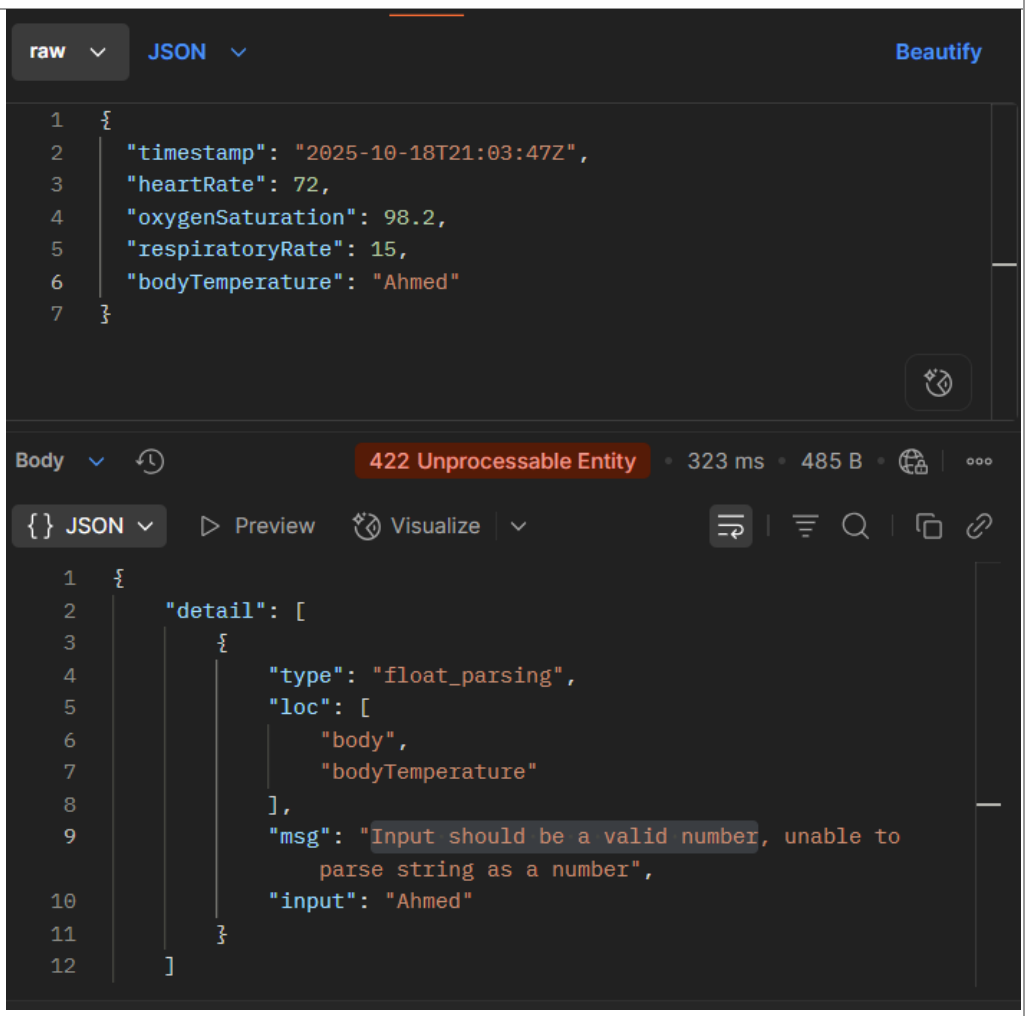
Test Case: Valid ingestion request

Table 6: Valid ingestion request

Test Case ID	UTC-API-03
Requirement	FR-ING-01
Preconditions	API running; auth token available.
Steps	1) POST valid metrics JSON. 2) Check response and DB.
Expected Result	{ "id": "<unique_mongodb_id>", "message": "Health data stored successfully" }
Actual Result	
Pass/Fail	Pass

Test Case: Invalid ingestion rejected

Table 7: Invalid ingestion rejected

Test Case ID	UTC-API-04
Requirement	FR-ING-02
Preconditions	API running.
Steps	1) POST malformed JSON or missing fields.
Expected Result	422 with invalid input message; nothing written to DB.
Actual Result	
Pass/Fail	Pass

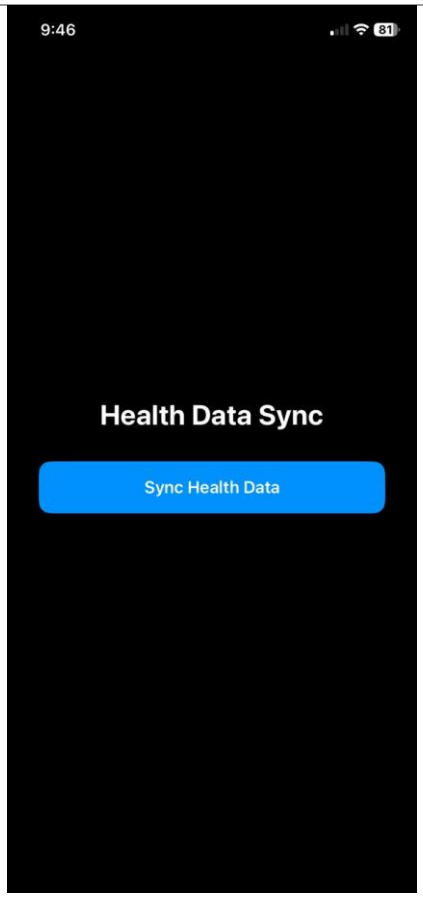
Test Case: iOS app builds in Xcode

Table 8: iOS app builds in Xcode

Test Case ID	UTC-IOS-01
Requirement	FR-IOS-01
Preconditions	Xcode project configured; signing set.
Steps	1) Build for device.
Expected Result	Build succeeds; installable app produced.
Actual Result	

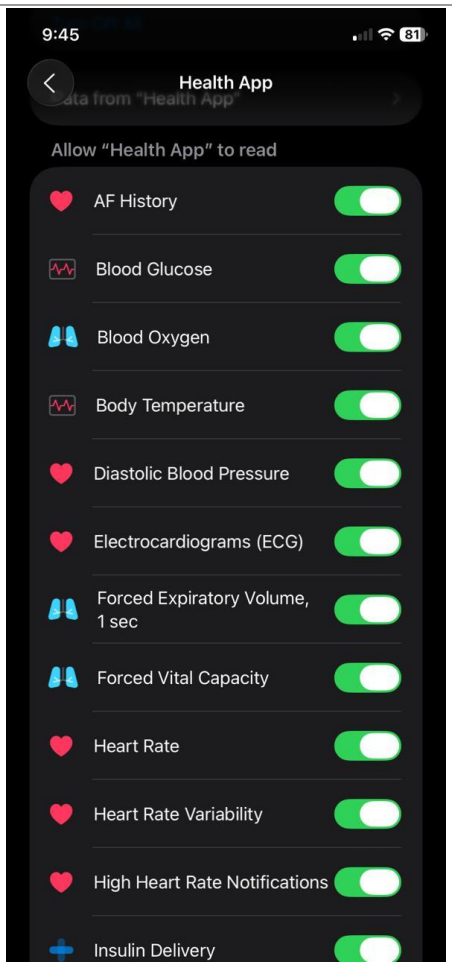
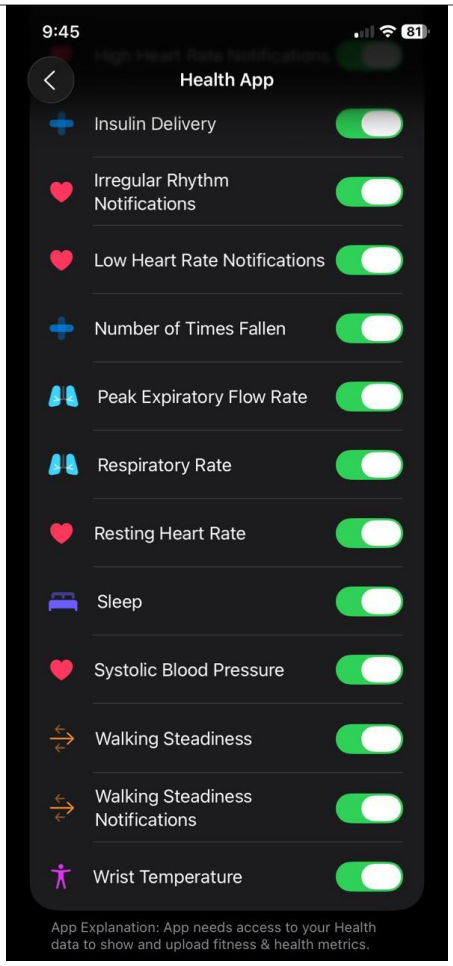
Test Case: App installs on iPhone

Table 9: App installs on iPhone

Test Case ID	UTC-IOS-02
Requirement	FR-IOS-02
Preconditions	Test device connected; build artifact ready.
Steps	1) Install from Xcode. 2) Launch app.
Expected Result	App installs and launches without crash.
Actual Result	
Pass/Fail	Pass

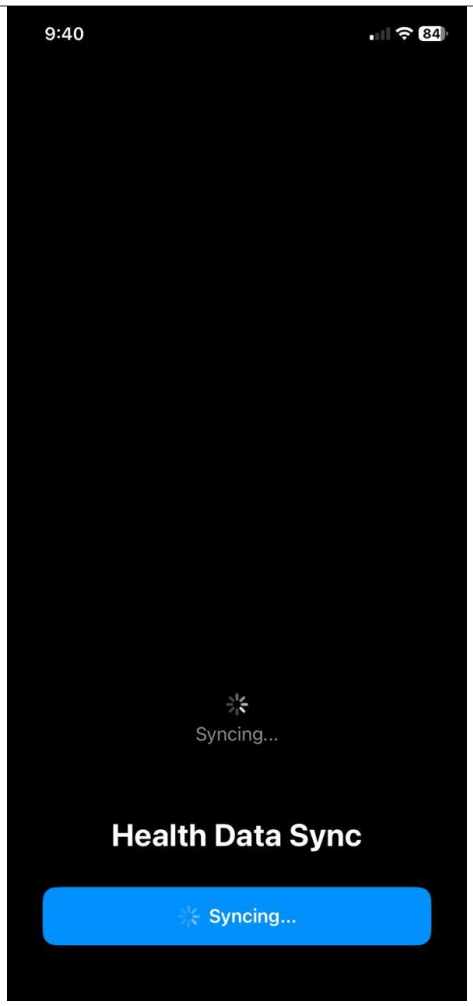
Test Case: Health permissions prompts

Table 10: Health permissions prompts

Test Case ID	UTC-IOS-03
Requirement	FR-PERM-01
Preconditions	Fresh install; Health data available.
Steps	1) Open app. 2) Proceed through permissions for HR, HRV, SpO_2, RR, temperature.
Expected Result	iOS shows prompts; selections saved; app reflects granted statuses.
Actual Result	 
Pass/Fail	Pass

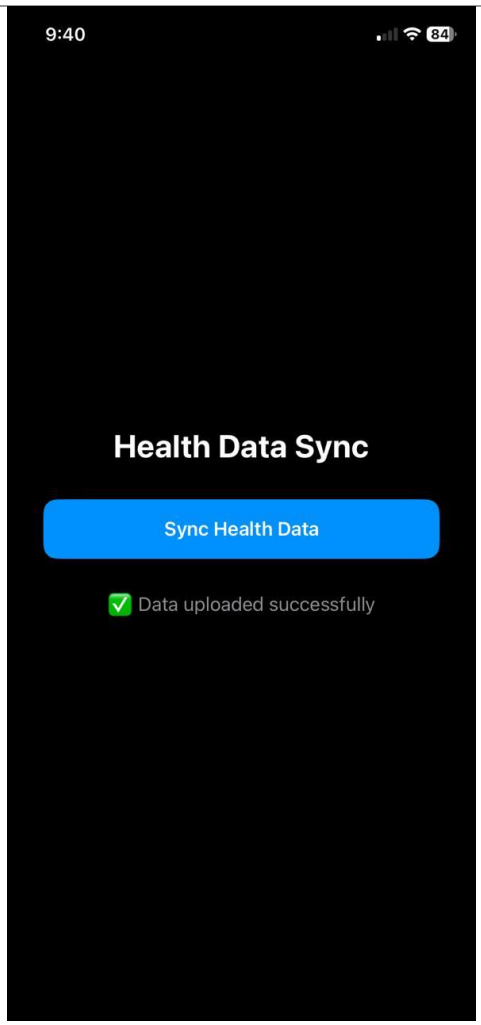
Test Case: Sync starts successfully

Table 11: Sync starts successfully

Test Case ID	UTC-SYNC-01
Requirement	FR-SYNC-01
Preconditions	Permissions granted; samples exist.
Steps	1) Open app; trigger sync. 2) Observe status/logs.
Expected Result	App indicates “sync started”; network calls initiated.
Actual Result	
Pass/Fail	Pass

Test Case: Sync completes successfully

Table 12: Sync completes successfully

Test Case ID	UTC-SYNC-02
Requirement	FR-SYNC-02
Preconditions	Active network; ingestion endpoint running.
Steps	1) Allow sync to finish. 2) Verify anchors updated.
Expected Result	App shows “sync completed”; batch acknowledged by API.
Actual Result	
Pass/Fail	Pass

Test Case: Data visible in MongoDB

Table 13: Data visible in MongoDB

Test Case ID	UTC-DB-02
Requirement	FR-DB-02
Preconditions	Successful sync occurred.
Steps	1) Query metrics by student mobile ID and time.
Expected Result	Records present with correct fields, units, timestamps.
Actual Result	QUERY RESULTS: 81-82 OF 82 _id: ObjectId('6901119aeb69b2e9f363b2aa') timestamp : "2025-10-28T18:55:22Z" heartRate : 60 restingHeartRate : 65 heartRateVariabilitySDNN : 42.22581776210951 oxygenSaturation : null respiratoryRate : null bodyTemperature : null irregularHeartRhythmEvent : null highHeartRateEvent : null lowHeartRateEvent : null ecgClassification : null ecgAverageHeartRate : null sleepAnalysisValue : null appleSleepingWristTemperature : null numberOfTimesFallen : 1 atrialFibrillationBurden : null bloodPressureSystolic : null bloodPressureDiastolic : null
Pass/Fail	Pass

Sprint-01 Review and Retrospective

What Went Well

- The iPhone app installed on a real device, requested permissions, and sent health data.
- The API deployed to the server, validated payloads, and wrote records to MongoDB.
- End-to-end flow was verified with real readings visible in the database.
- Basic logging and simple metrics helped the team confirm status quickly.

What Could Be Improved

- Background delivery showed occasional delay during poor network conditions.
- Error messages for denied permissions need to be clearer for students.
- Duplicate prevention after app restart needs more test coverage.
- Monitoring for offline queue length and retry results is limited.

What We Will Try Next

- Add data quality checks and baseline summaries to prepare for the alert model.
- Tighten payload validation and expand negative test cases.
- Refine the permission screens and in-app guidance.
- Improve observability with simple dashboards for API rate, failures, and database writes.

Sprint-01 Velocity

- Completed User Stories: 4
- Sprint Duration: 14 days
- Sprint Velocity: 4 user stories completed in a 14-day sprint

Sprint-02 Overview

Sprint 2 focused on improving alert accuracy, strengthening the doctor decision workflow, and adding clarity to the alert data presented in the portal. This sprint refined the analytics layer, improved the reliability of status changes, and ensured alerts carry contextual reasoning to support doctors' decisions.

Sprint-02 Goal

Enhance the quality and clarity of alerts by improving trend analysis, refining thresholds, and ensuring the doctor portal displays information that supports reliable accept or reject decisions.

User Stories Implemented

- As a doctor, I want alerts to trigger only when reliable analytics confirm an abnormal pattern so that I can trust the notifications I receive.
- As a doctor, I want each alert to include clear context and reasoning so that I can make an informed acceptance or rejection decision.
- As a system administrator, I want every doctor's decision recorded in a structured log so that the system maintains traceable accountability

Tasks Implemented

- Refined threshold logic for heart rate, HRV, SpO2, temperature and respiratory rate.
- Added trend analysis to avoid false positives based on single readings.
- Implemented reasoning metadata attached to each alert.
- Updated doctor portal alert detail view to show trend graphs and key values.
- Strengthened backend logic for accept and reject workflows.
- Added structured logging for all alert decisions.
- Ensured status transitions cannot skip states or enter invalid flow.

Functional Requirements

Alert Precision

- System shall trigger alerts only when consistent abnormal readings are detected.
- System shall evaluate trends using multiple samples rather than single values.
- System shall attach reasoning metadata to every alert.

Doctor Workflow

- System shall display detailed context for each alert in the doctor portal.
- System shall support accept and reject actions through a valid and traceable workflow.
- System shall log every doctor's decision with timestamp, doctor ID, and case ID.

System Integrity

- System shall enforce validated status transition rules for all status changes.
- System shall log failed decision attempts for debugging purposes.

Class Diagram: Sprint 1

Class diagram showing all core components of the student health monitoring system, including domain entities, device and app elements, backend services, alert processing logic, doctor workflow, and notification flow. It illustrates how readings move from the Apple Watch through the mobile app into the backend, how alerts are generated and routed to doctors, how decisions are recorded, and how communication reaches students.

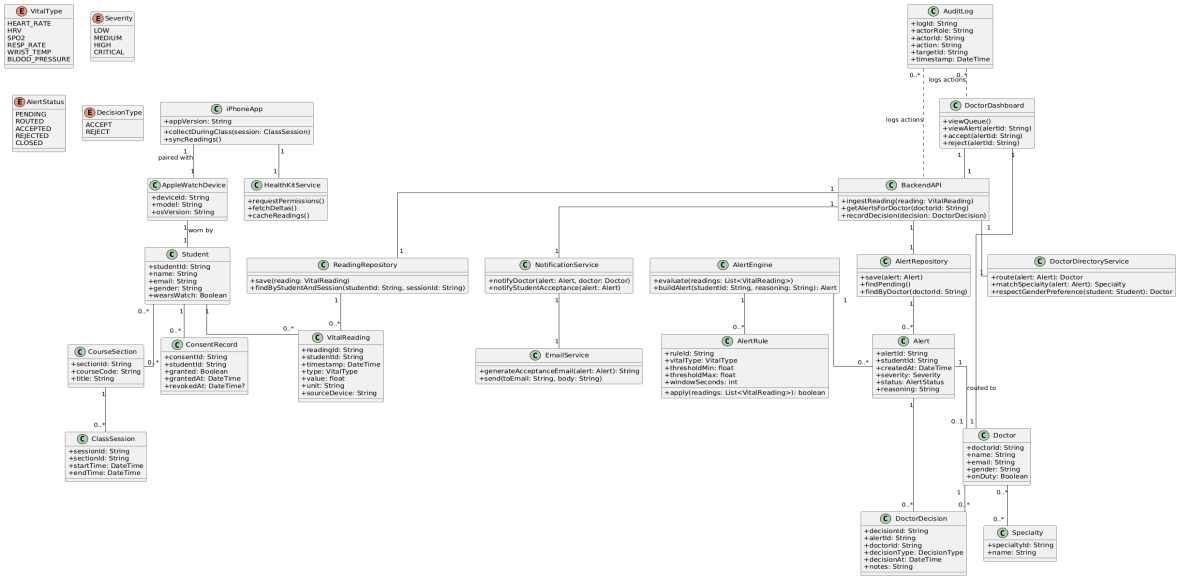


Figure 4: Class Diagram: Sprint 1

Activity Diagrams of Sprint 2

Flow 1 : Process showing how Apple Watch readings are collected, cached when needed, and uploaded through the iPhone app to the backend, where they are validated and stored.

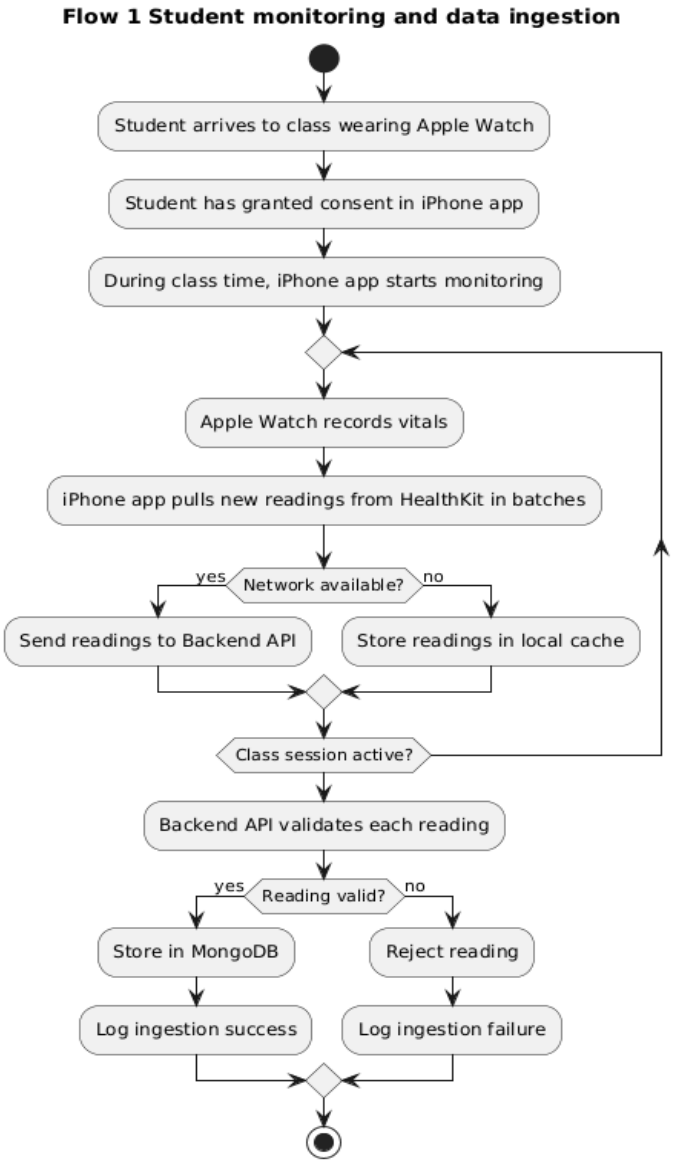


Figure 5: Figure 4: Activity Diagram-Flow 1

Flow 2: Process showing how the backend groups readings, evaluates them with rules, generates alerts when abnormal patterns appear, and routes the alerts to suitable doctors.

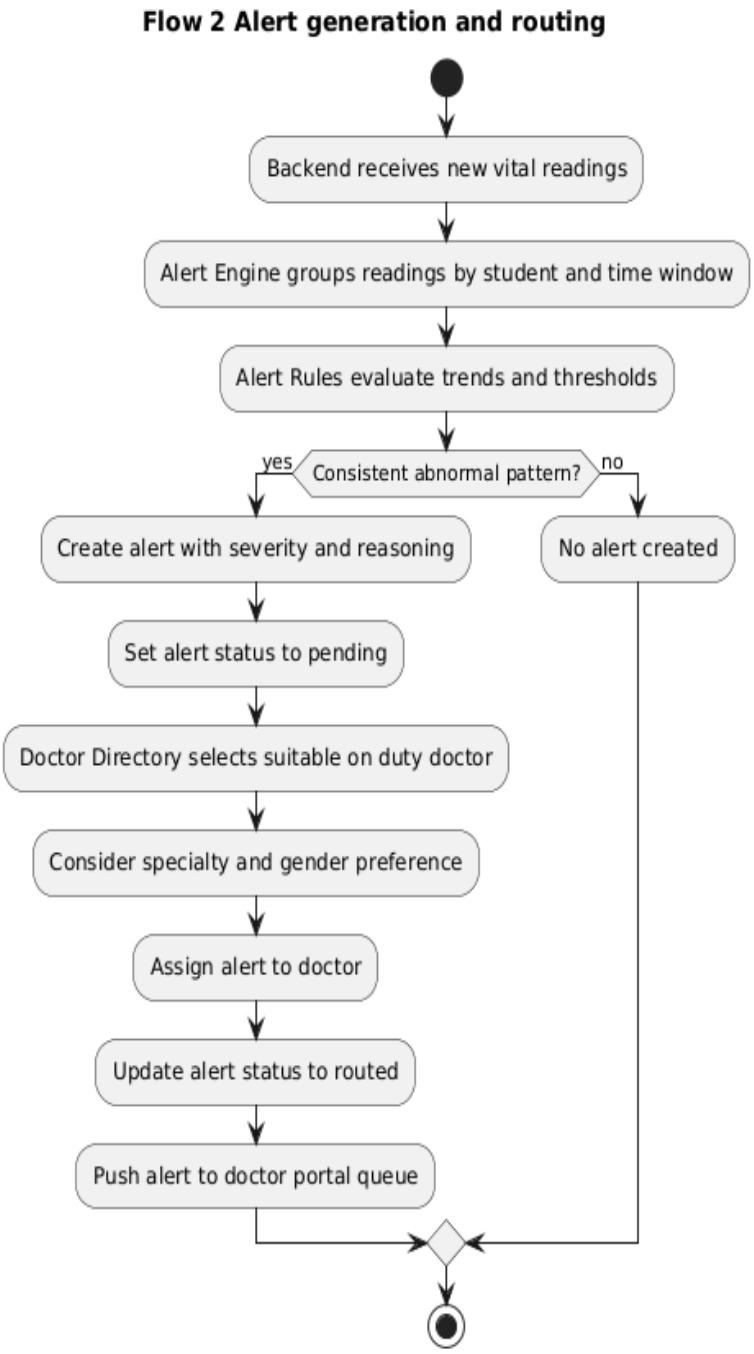


Figure 6: Activity Diagram-Flow 2

Flow 3 Process showing how doctors review routed alerts, inspect the details, and decide to accept or reject while the system records decisions and maintains audit history.

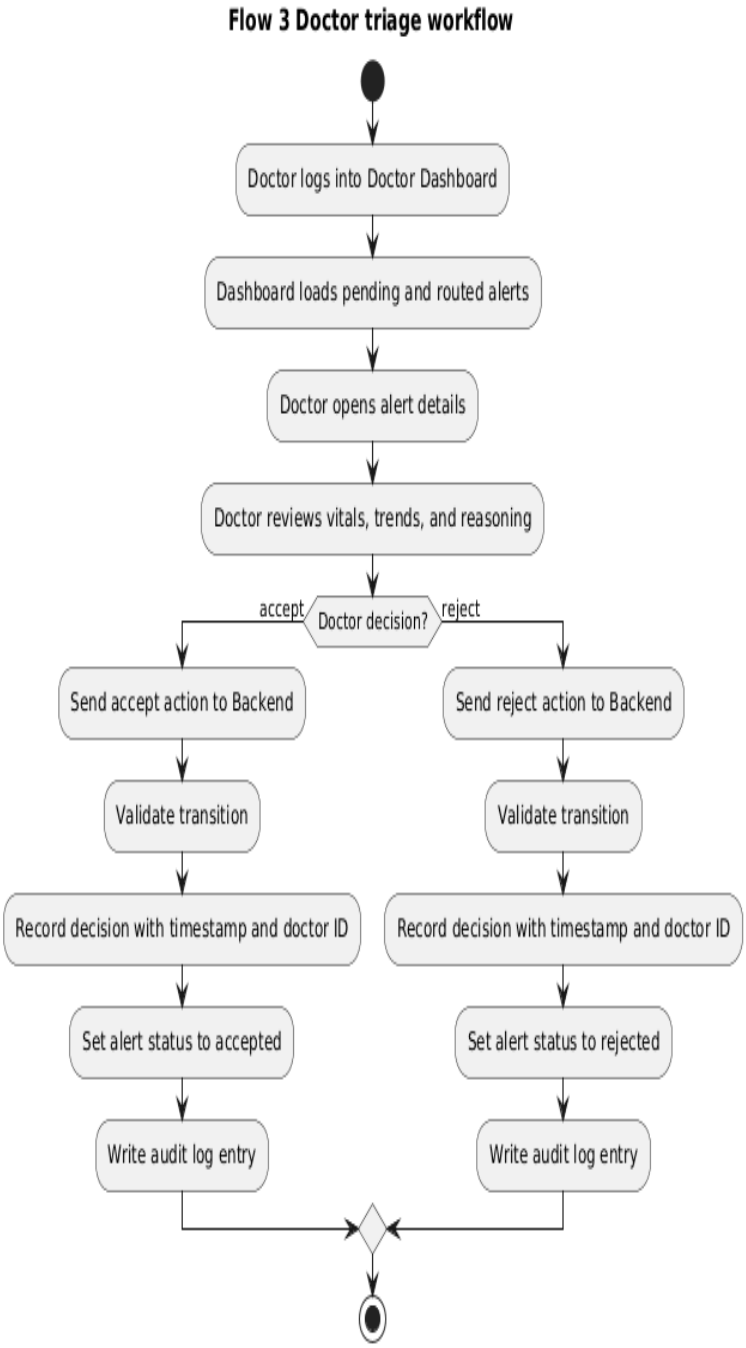


Figure 7: Activity Diagram-Flow 3

Flow 4 Process showing how an accepted alert triggers student communication through a generated guidance email, with delivery checks and retry handling.

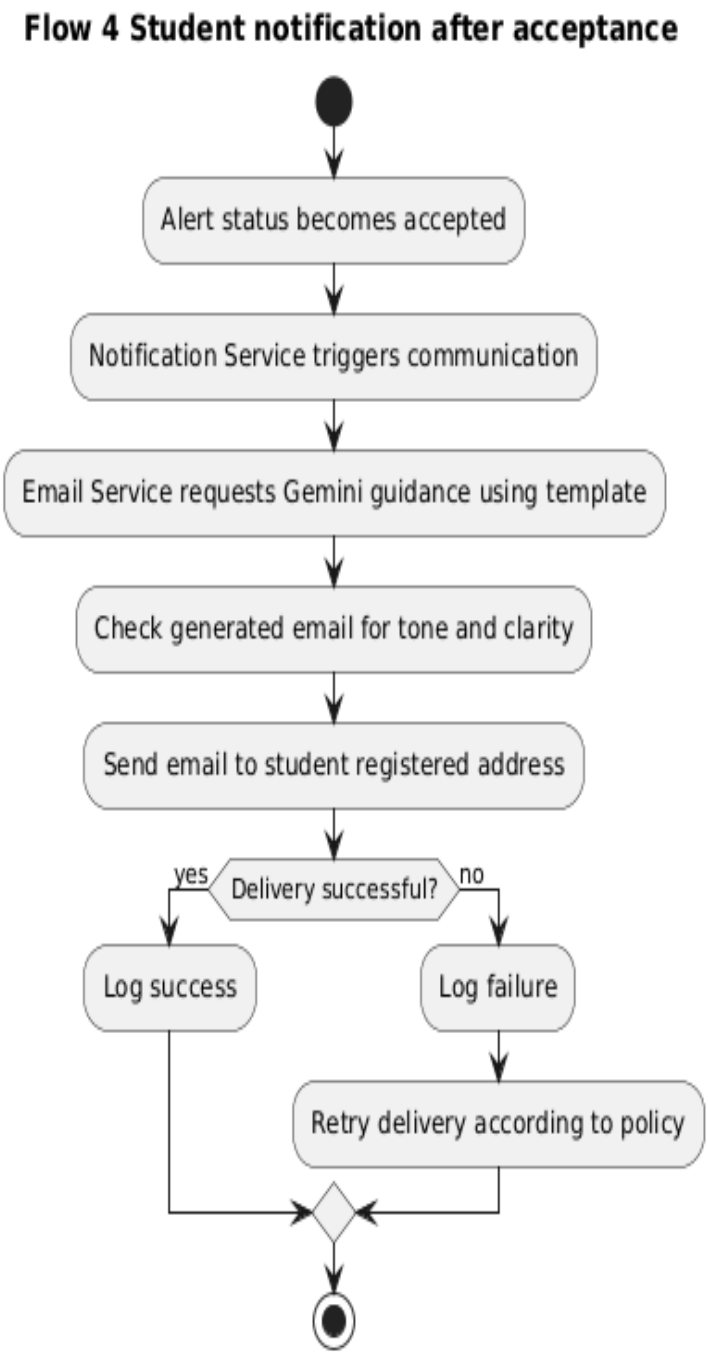


Figure 8: Activity Diagram-Flow 4

Flow 5 Process showing the complete classroom safety cycle from continuous monitoring to alert generation, doctor triage, student notification, and KPI tracking.

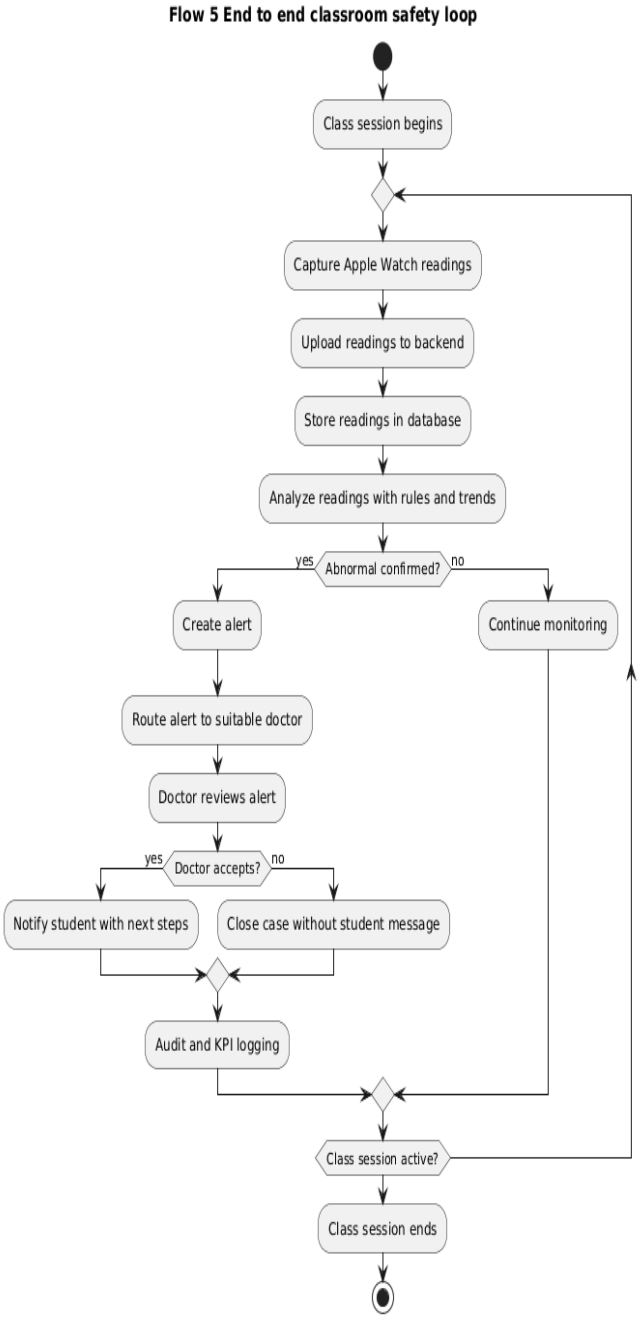
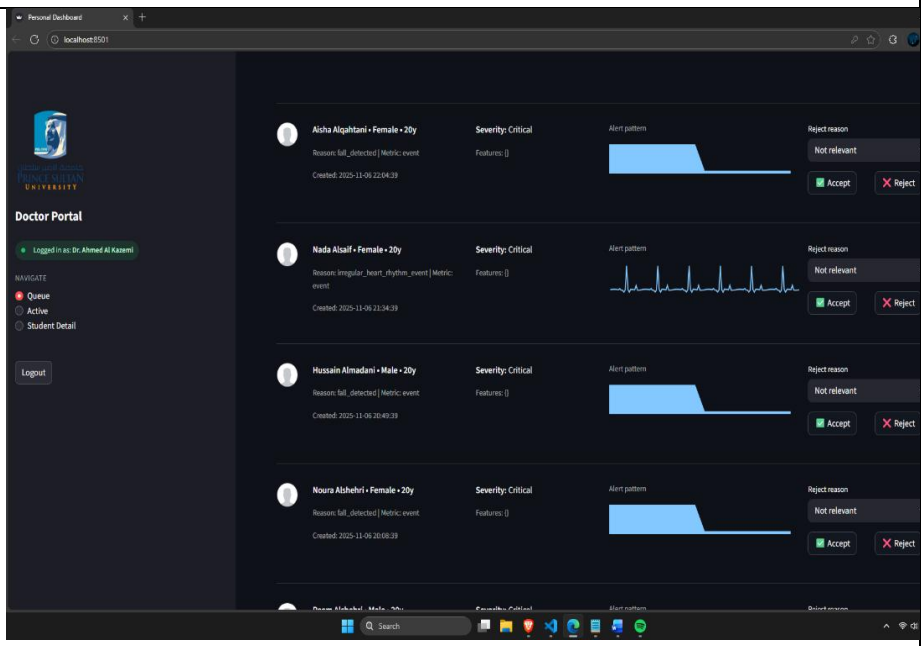


Figure 9: Activity Diagram-Flow 5

Unit Test Cases

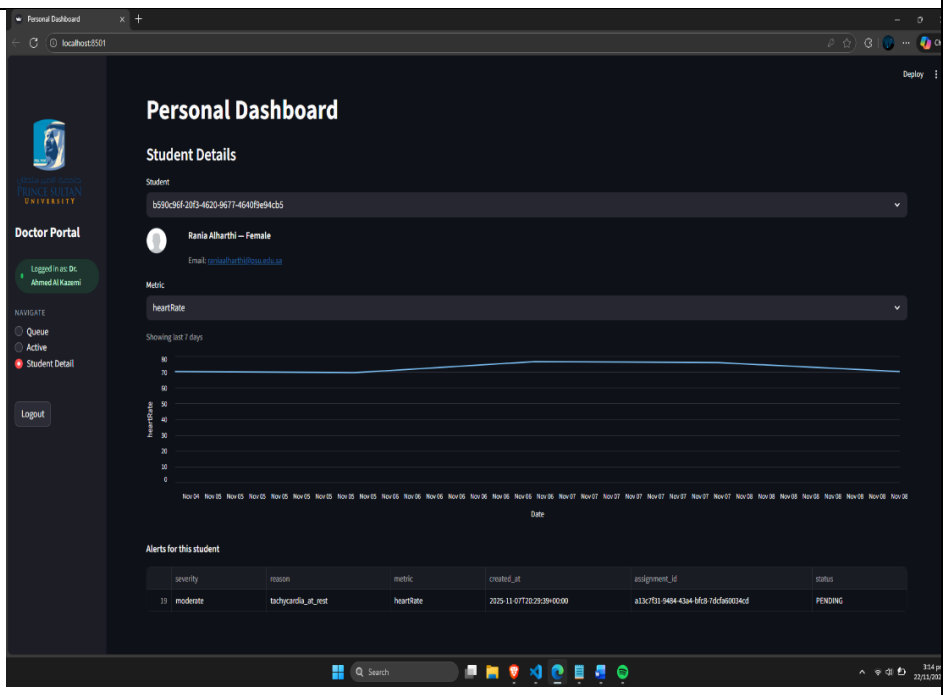
Test Case: UTC-ALERT-01

Table 14: UTC-ALERT-01

Test Case ID	**UTC-ALERT-01**
Requirement ID	FR-ALERT-01
Preconditions	The student has valid consent. The database contains continuous readings.
Steps	1) Insert slightly abnormal readings. 2) Run alert evaluation. 3) Insert sustained abnormal readings. 4) Re-evaluate.
Expected Result	Alert only after sustained abnormality.
Actual Result	
Pass/Fail	Pass

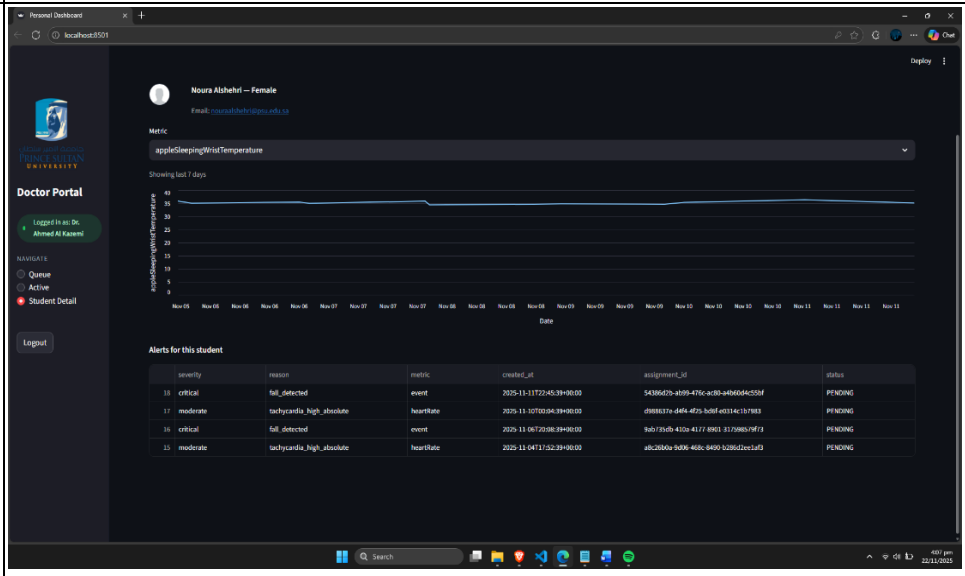
Test Case: UTC-ALERT-02

Table 15: UTC-ALERT-02

Test Case ID	**UTC-ALERT-02**
Requirement ID	FR-ALERT-02
Preconditions	Alert engine active. Thresholds set.
Steps	1) Insert normal readings. 2) Insert one abnormal reading. 3) Continue normal readings.
Expected Result	No alert created for single spike.
Actual Result	
Pass/Fail	Pass

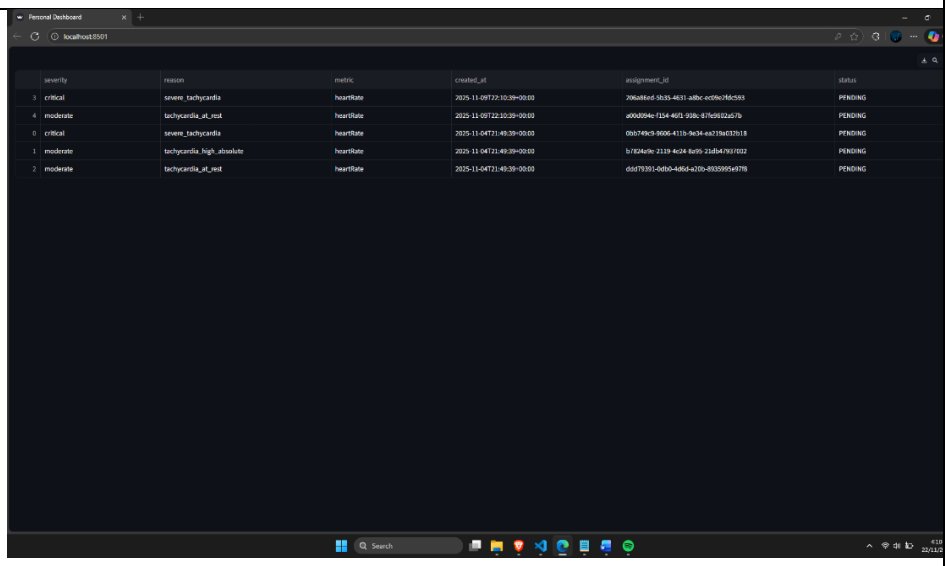
Test Case: UTC-ALERT-03

Table 16: UTC-ALERT-03

Test Case ID	**UTC-ALERT-03**
Requirement ID	FR-ALERT-03
Preconditions	Severity rules configured.
Steps	1) Insert moderate abnormal readings 2) Insert critical abnormalities.
Expected Result	Severity correctly assigned.
Actual Result	
Pass/Fail	Pass

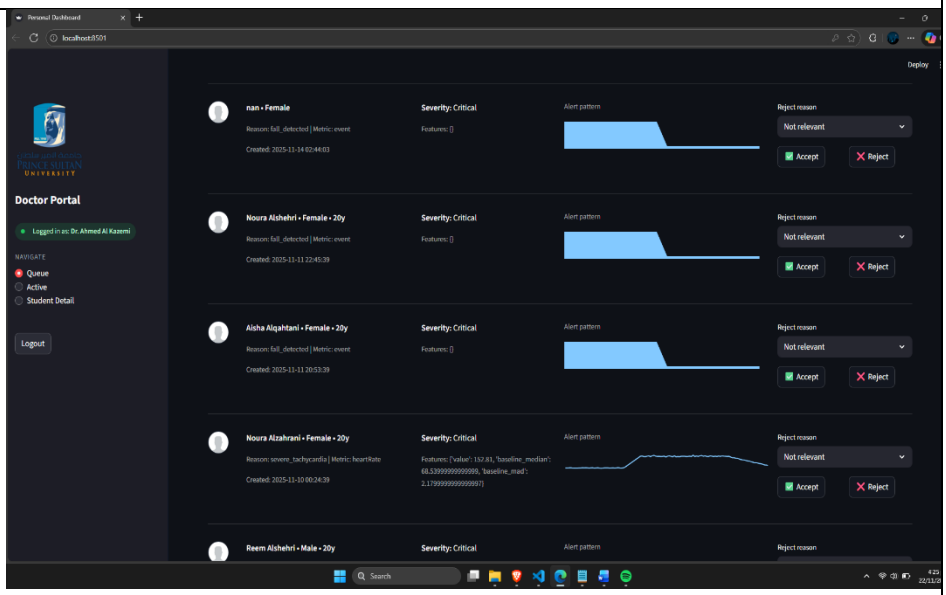
Test Case: UTC-ALERT-04

Table 17: UTC-ALERT-04

Test Case ID	**UTC-ALERT-04**
Requirement ID	FR-ALERT-04
Preconditions	Alert engine stores reasoning.
Steps	1) Trigger alert. 2) Fetch alert from DB.
Expected Result	Alert includes reasoning metadata.
Actual Result	
Pass/Fail	Pass

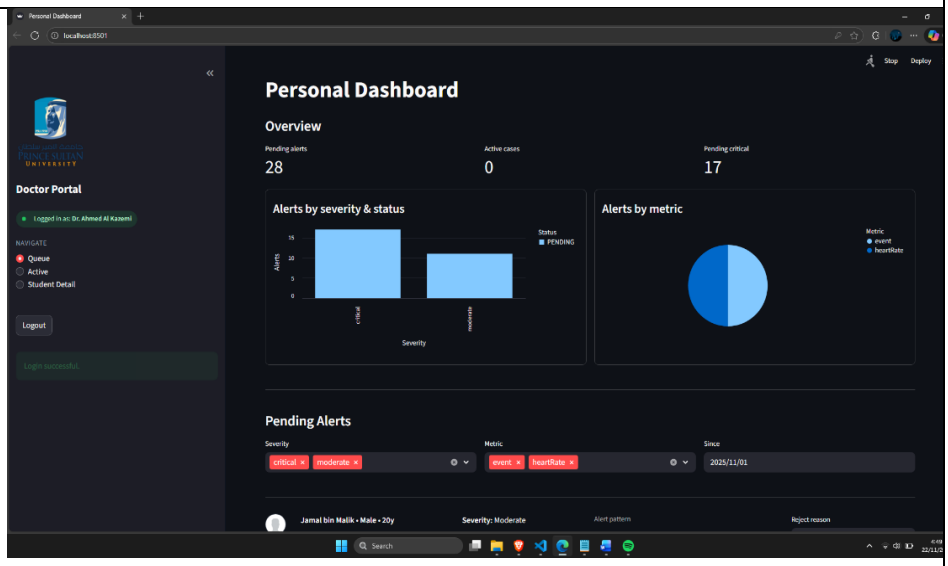
Test Case: UTC-WF-01

Table 18: UTC-WF-01

Test Case ID	**UTC-WF-01**
Requirement ID	FR-WF-01
Preconditions	Backend connected to repository.
Steps	1) Trigger alert. 2) Read alert status.
Expected Result	Initial state is Pending.
Actual Result	
Pass/Fail	Pass

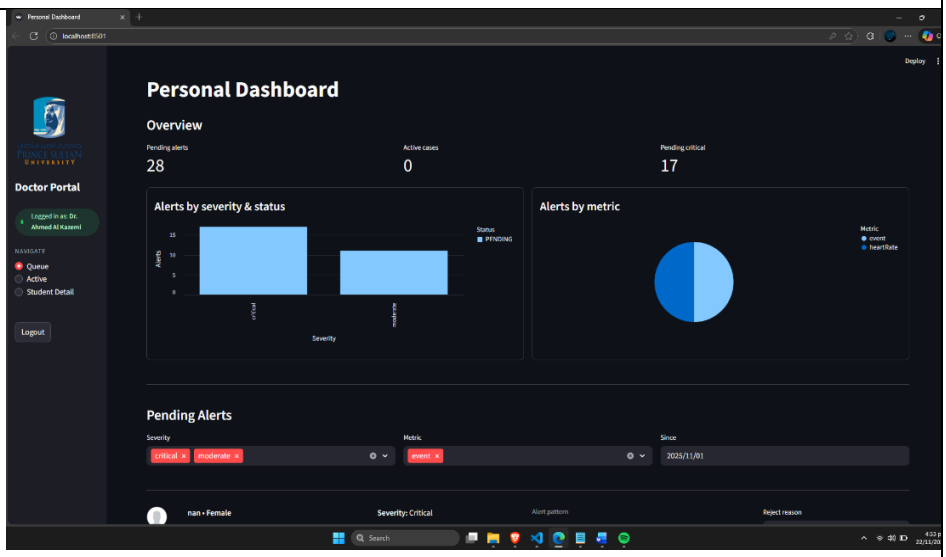
Test Case: UTC-Log In-01

Table 19: UTC-Log In-01

Test Case ID	UTC-Log In-01
Requirement ID	FR-LogIn-01
Preconditions	Directory contains doctors.
Steps	1) Try Signing in with Correct Password and Email
Expected Result	Signs In.
Actual Result	
Pass/Fail	Pass

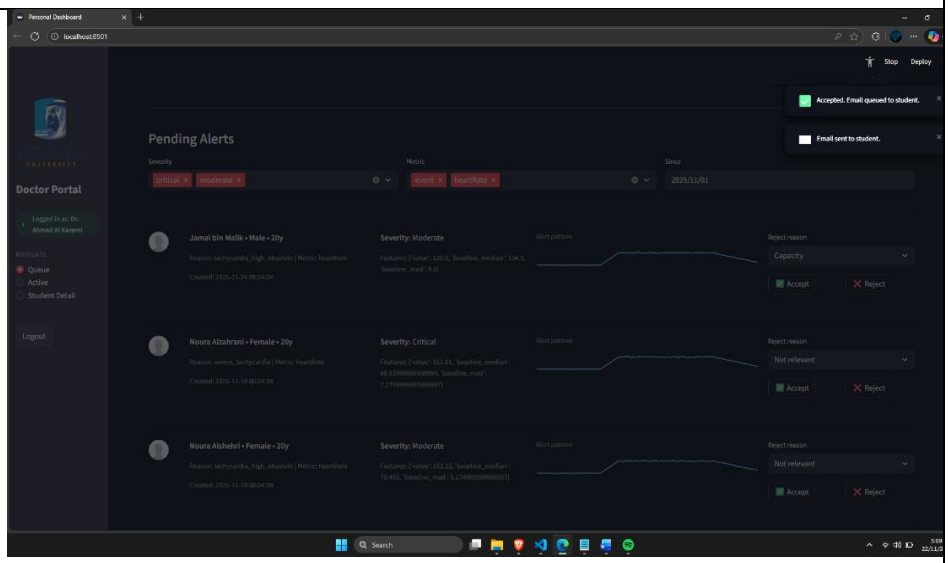
Test Case: UTC-UI-01

Table 20: UTC-Log In-01

Test Case ID	**UTC-UI-01**
Requirement ID	FR-UI-01
Preconditions	Doctor authenticated.
Steps	1) Open doctor queue. 2) Select alert.
Expected Result	Detail page shows trends and reasoning.
Actual Result	
Pass/Fail	Pass

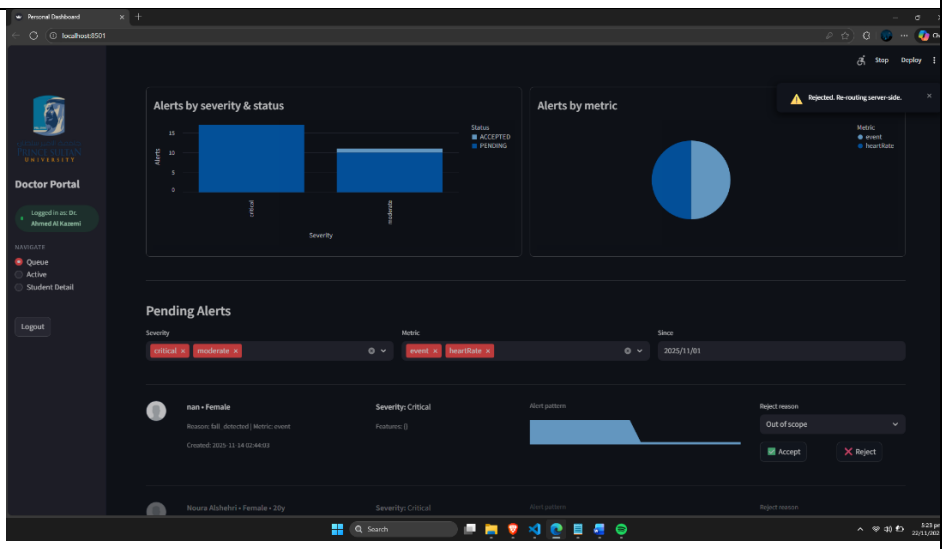
Test Case: UTC-DEC-01

Table 21: UTC-DEC-01

Test Case ID	**UTC-DEC-01**
Requirement ID	FR-DEC-01
Preconditions	Alert routed.
Steps	1) Doctor accepts. 2) System logs decision.
Expected Result	Status becomes Accepted.
Actual Result	
Pass/Fail	Pass

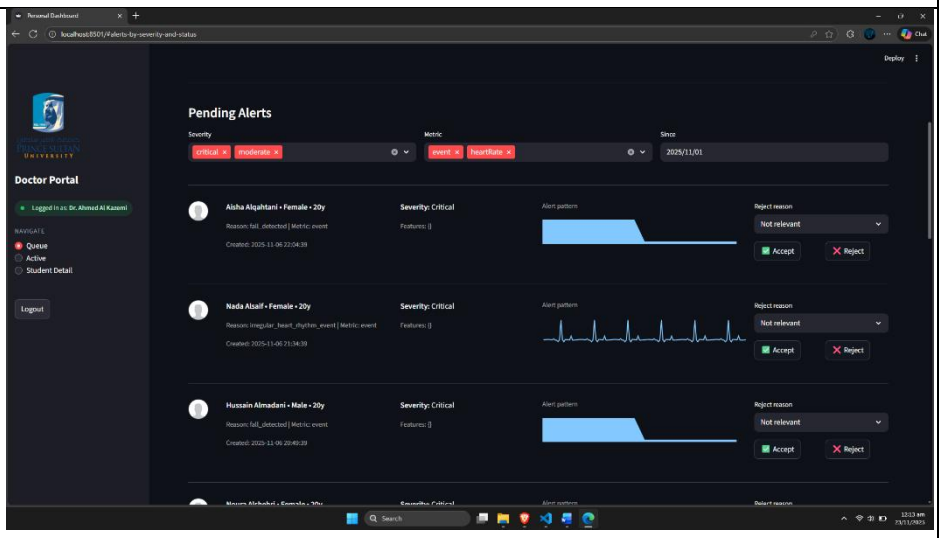
Test Case: UTC-DEC-02

Table 22: UTC-DEC-02

Test Case ID	**UTC-DEC-02**
Requirement ID	FR-DEC-02
Preconditions	Alert routed.
Steps	1) Doctor rejects. 2) System logs decision.
Expected Result	Status becomes Rejected.
Actual Result	
Pass/Fail	Pass

Test Case: UTC-WF-02

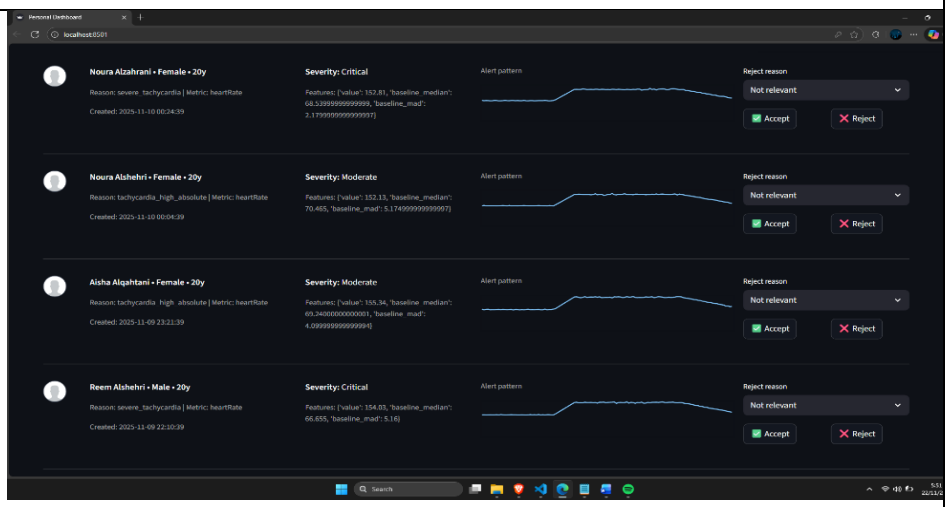
Table 23: UTC-WF-02

Test Case ID	UTC-WF-02
Requirement ID	FR-WF-02
Preconditions	Alert Generated
Steps	1.Try to change rejection reason
Expected Result	Doctor can change the rejection reason.
Actual Result	
Pass/Fail	Fail

Integration Test Cases

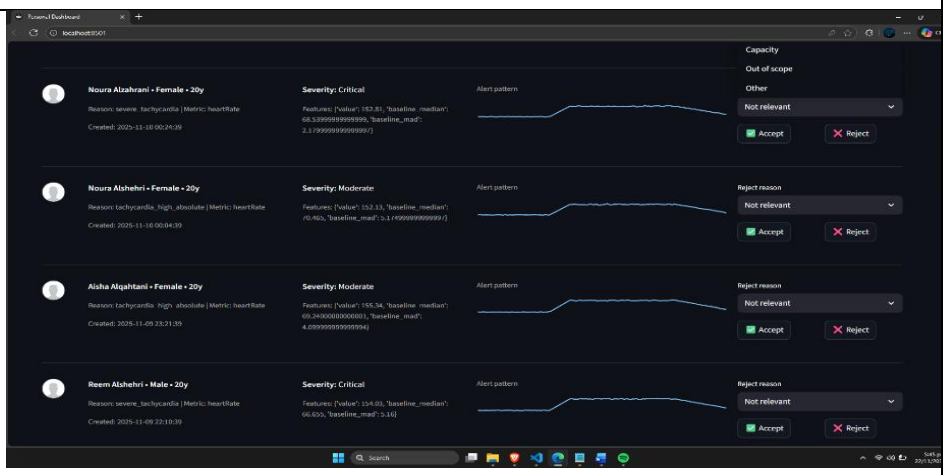
Test Case: ITC-E2E-01

Table 24: UTC-WF-02

Test Case ID	**ITC-E2E-01**
Requirement ID	IR-E2E-01
Preconditions	End to end system running.
Steps	1) Send normal readings. 2) Send abnormal readings. 3) Wait. 4) Check doctor portal.
Expected Result	Alert appears correctly routed.
Actual Result	
Pass/Fail	Pass

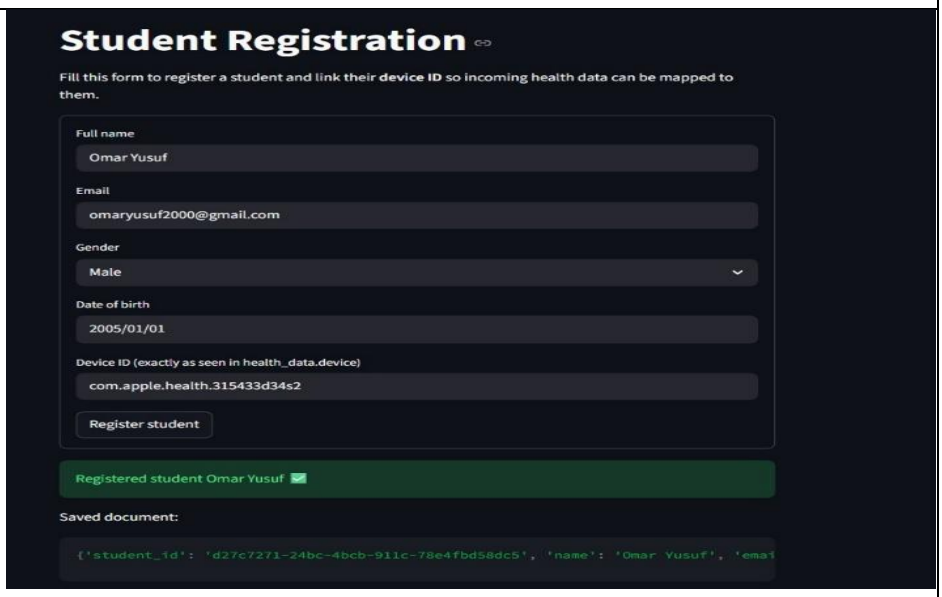
Test Case: ITC-E2E-02

Table 25: ITC-E2E-02

Test Case ID	**ITC-E2E-02**
Requirement ID	IR-E2E-02
Preconditions	Routed alert exists.
Steps	1) Doctor accepts. 2) Refresh portal. 3) Check DB.
Expected Result	Decision reflected everywhere.
Actual Result	
Pass/Fail	Pass

Test Case: ITC-Student Registration-01

Table 26: ITC-Student Registration-01

Test Case ID	**ITC-Student Registration-01**
Requirement ID	ITC-Student Registration-01
Preconditions	Device ID exists
Steps	1. Fill in the student's details. 2. Submit the form
Expected Result	Student information is saved
Actual Result	
Pass/Fail	Pass

Sprint-02 Review and Retrospective

What Went Well

- Alert quality improved significantly.
- Doctor workflow became clearer and more structured.
- Trend based analytics reduced false positives.

What Could Be Improved

- Portal visuals could benefit from clearer graphs.
- Some reasoning metadata could be more readable.

What We Will Try Next

- Smoother integration of trend charts.
- Improving readability of reasoning metadata.
- Preparing email workflow for Sprint 3.

Sprint Velocity

- Stories completed: 3
- Duration: 2 weeks
- Velocity consistent with team expectation.

Sprint-03 Overview

Sprint 3 focused on strengthening communication reliability and ensuring that students receive clear guidance messages after doctor acceptance. This sprint introduced Gemini-generated email content, improved performance under realistic loads, and added monitoring and failure logging to the backend services.

Sprint Goal

Deliver dependable student notification through high-quality Gemini-generated emails while validating system performance, stability, and error handling under increased operational load.

User Stories Implemented

- As a student, I want to receive a clear guidance email whenever a doctor accepts my alert so that I know what to do next.
- As a doctor, I want the email sent to students to be consistent and properly written so that communication remains professional and trustworthy.
- As an operator, I want to monitor backend and alert engine performance so that failures, delays, or bottlenecks can be detected early.

Tasks Implemented

- Integrated Gemini template-based responses into the acceptance workflow.
- Implemented formatting and tone validation for generated email messages.
- Added Email Service retry logic for failed deliveries.
- Introduced detailed failure logging for email, AI, and queue operations.
- Added performance monitoring metrics for alert engine throughput.
- Improved backend optimizations for concurrent alert evaluations.
- Implemented backend alerts for slow or failed message generations.

Functional Requirements

Email Quality and Delivery

- System shall send an email to the assigned student for every accepted alert.
- System shall ensure Gemini-generated text matches the required tone and clarity guidelines.
- System shall log email delivery failures and retry delivery.

Performance and Monitoring

- System shall handle concurrent alert evaluations under simulated load.
- System shall capture backend performance metrics for analysis.
- System shall log failures in email generation, routing, or sending.

Workflow Consistency

- System shall immediately initiate the messaging chain upon doctor acceptance.
- System shall not complete any acceptance event without attempting a student notification.

Class Diagram: Sprint-03

Class diagram for Sprint 3 showing the acceptance to notification pipeline, including doctor decisions recorded through the backend, Gemini-based email content generation, email queuing and delivery with retry handling, plus monitoring and failure logging to ensure reliable student communication and system performance under load.

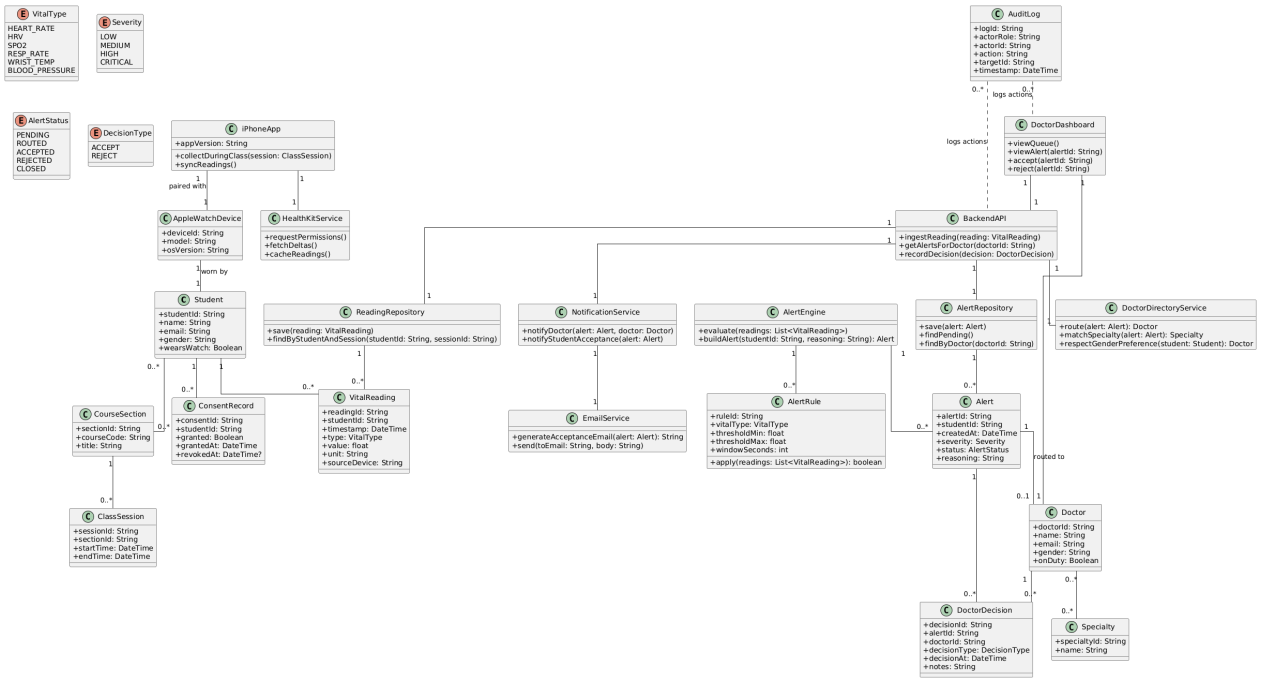


Figure 10: Class diagram for Sprint 3

Activity Diagrams of Sprint-03

Flow 1 Flow showing how a doctor's acceptance or rejection moves through backend validation, status updating, decision logging, and audit recording.

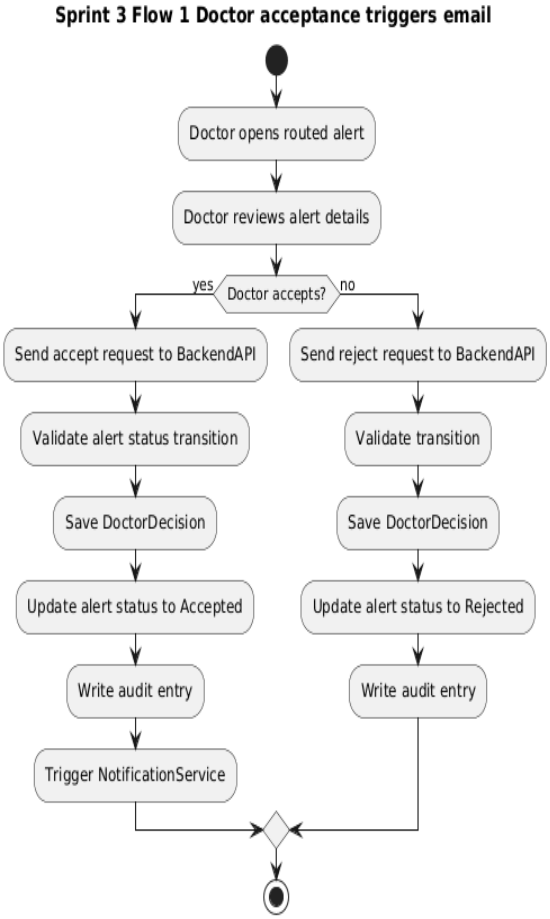


Figure 11: Activity diagram for Sprint 3-Flow 1

Flow 2 Flow showing how accepted alerts trigger Gemini-based guidance generation, email creation, queuing, delivery, and fallback handling when tone validation fails.

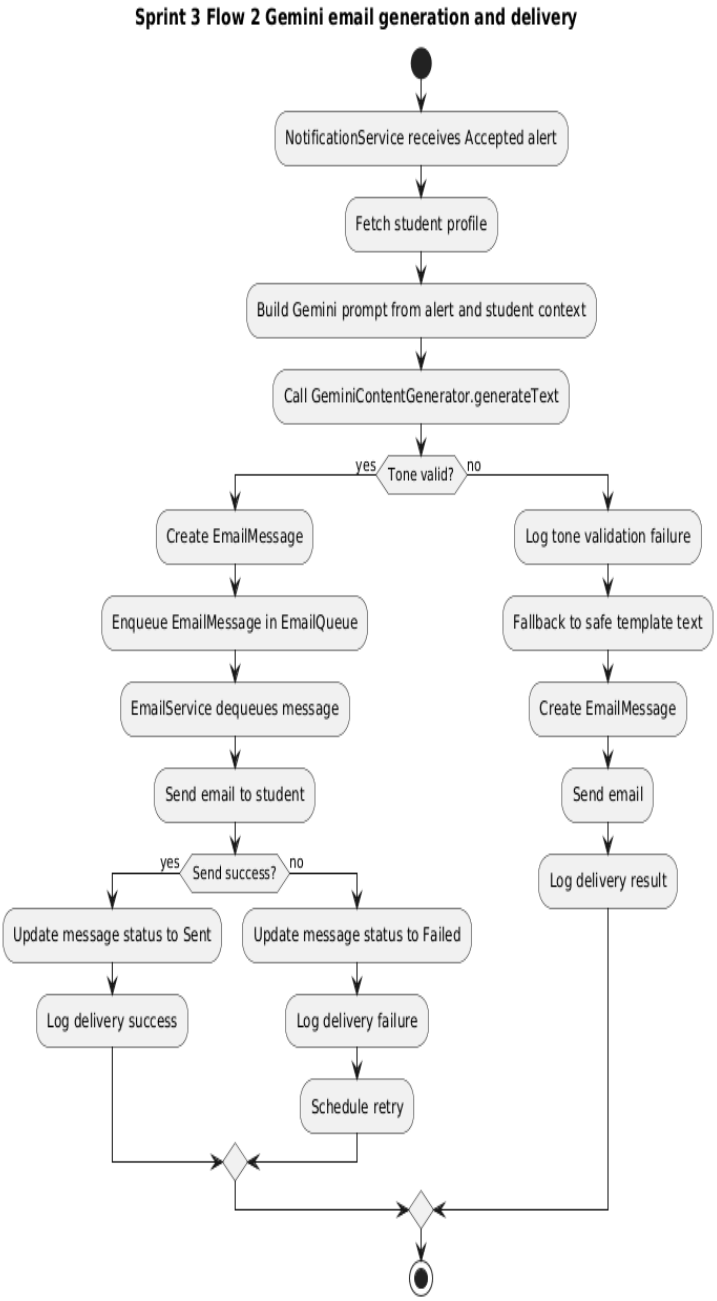


Figure 12: Activity diagram for Sprint 3-Flow 2

Flow 3 Flow showing email retry logic, including failure detection, retry scheduling, resend attempts, success updating, and escalation when retries are exhausted.

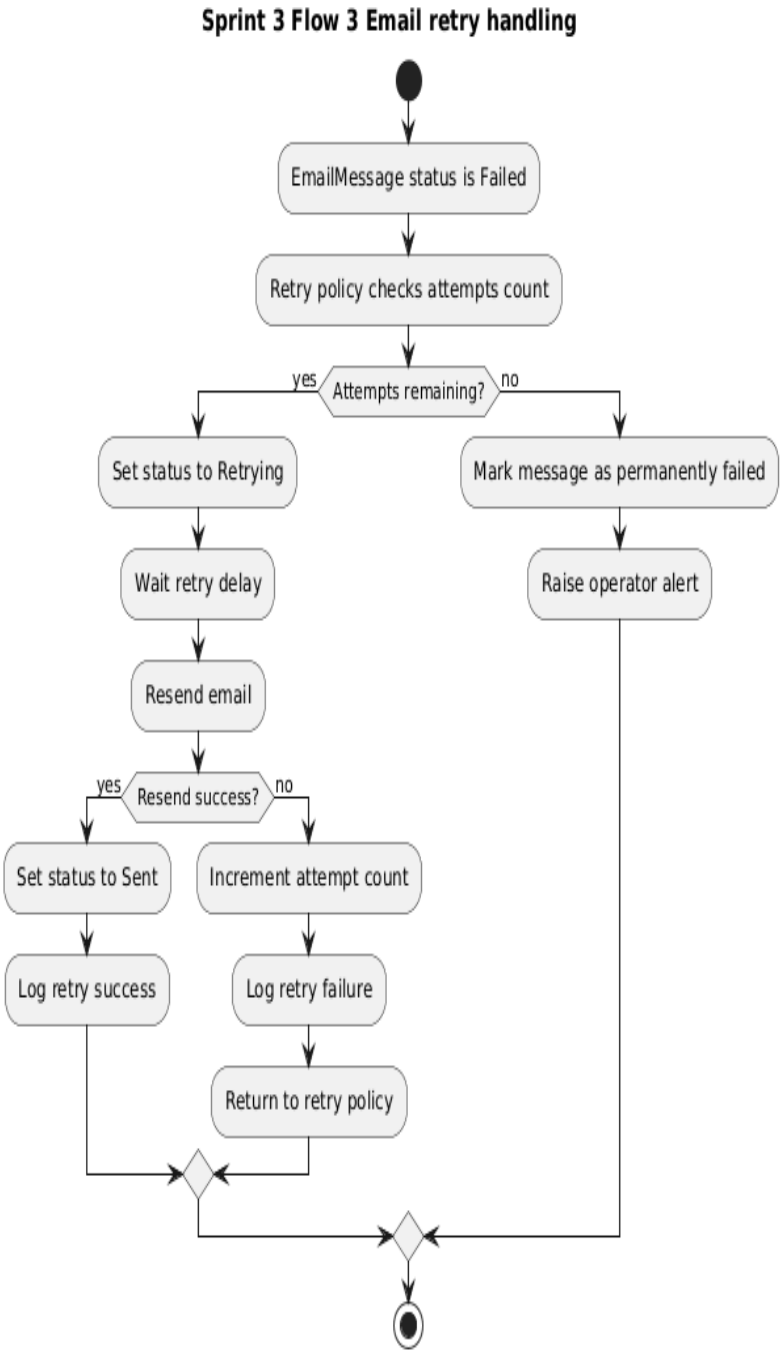


Figure 13: Activity diagram for Sprint 3-Flow 3

Flow 4 Flow showing system performance monitoring where latency, failures, and component issues are logged and aggregated into periodic metrics reports.

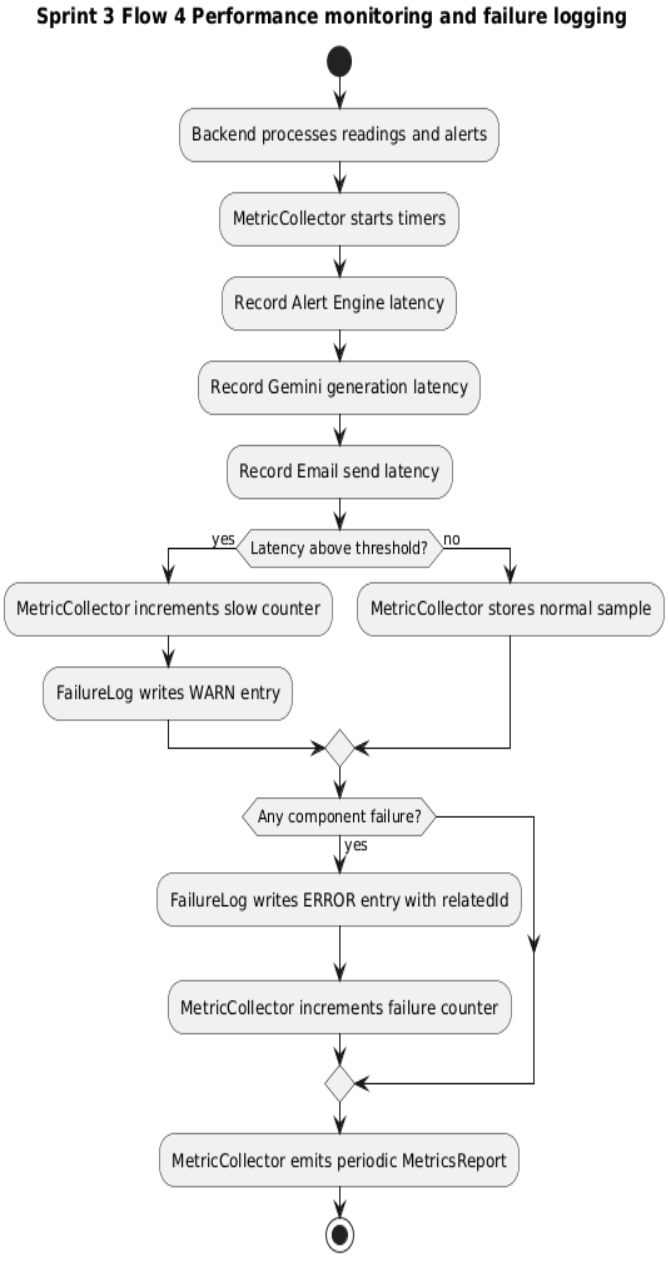


Figure 14: Activity diagram for Sprint 3-Flow 4

Flow 5 Flow showing the full Sprint 3 communication path from doctor acceptance through Gemini message creation, email delivery, retry handling, and final logging of success or failure.

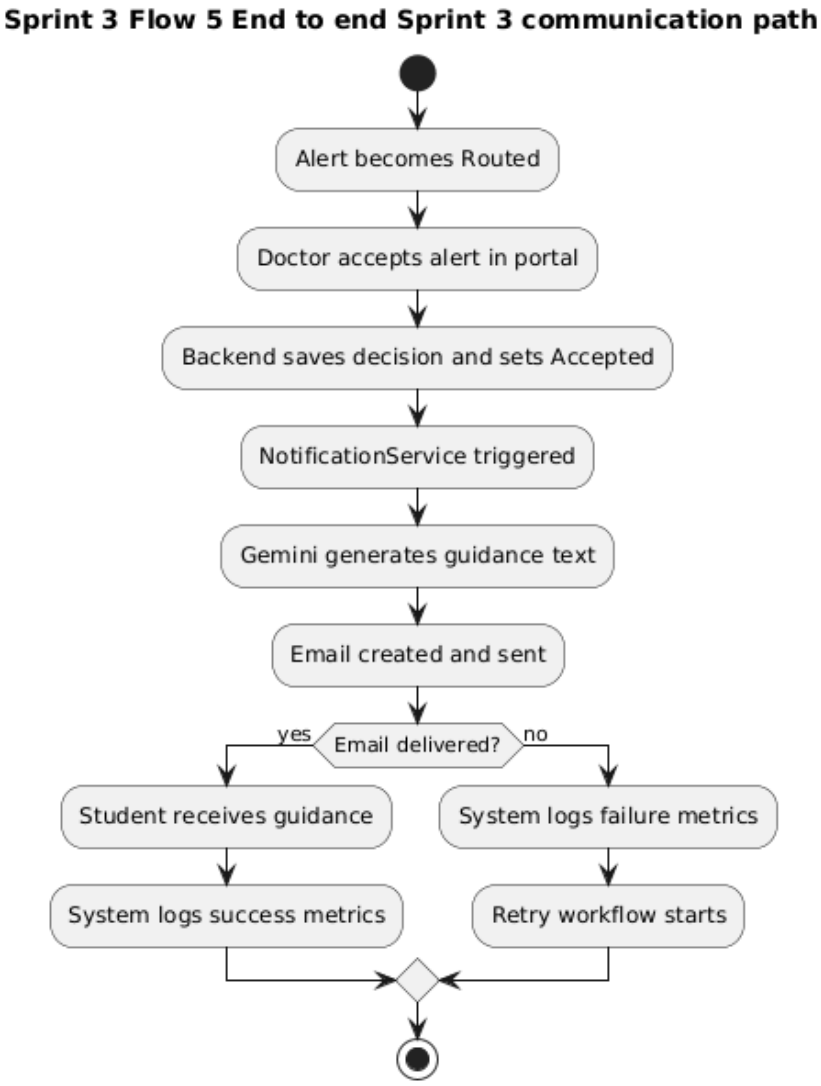
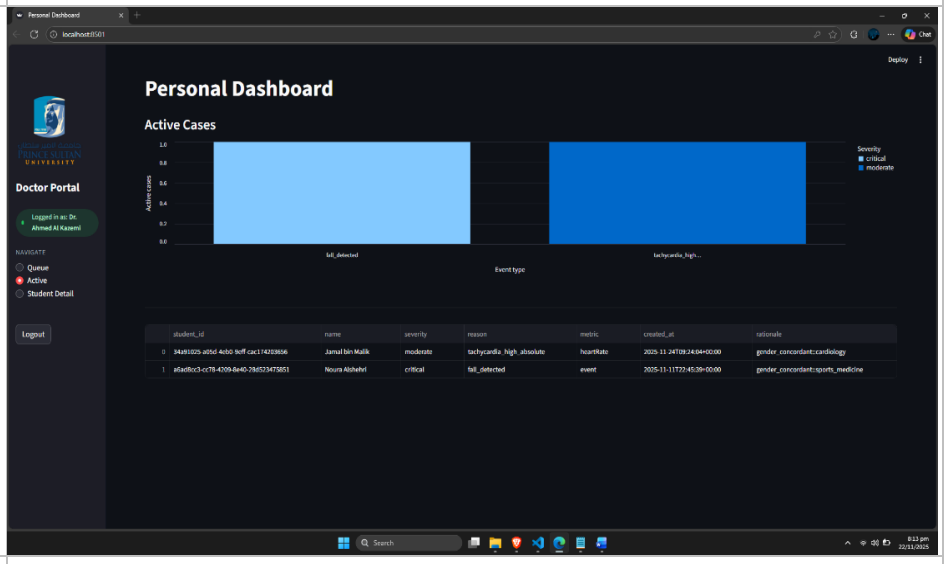


Figure 15: Activity diagram for Sprint 3-Flow 5

Unit Test Cases

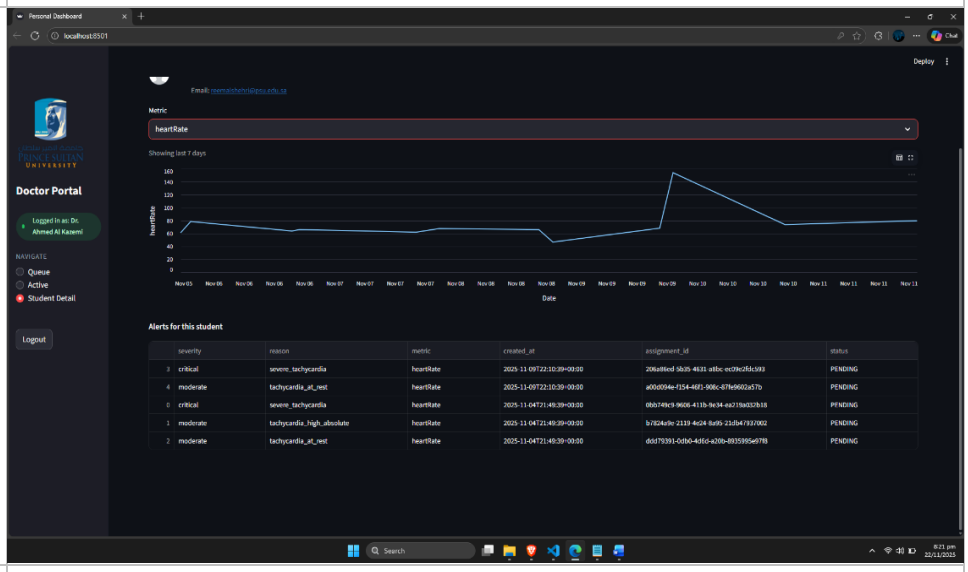
Test Case: UTC-FE-S3-01

Table 27: UTC-FE-S3-01

Test Case ID	**UTC-FE-S3-01**
Requirement ID	FE-S3-01
Preconditions	Doctor logged in, at least one routed alert exists
Steps	1) Open dashboard. 2) View alert list.
Expected Result	Routed alerts displayed with severity and timestamp.
Actual Result	
Pass/Fail	Pass

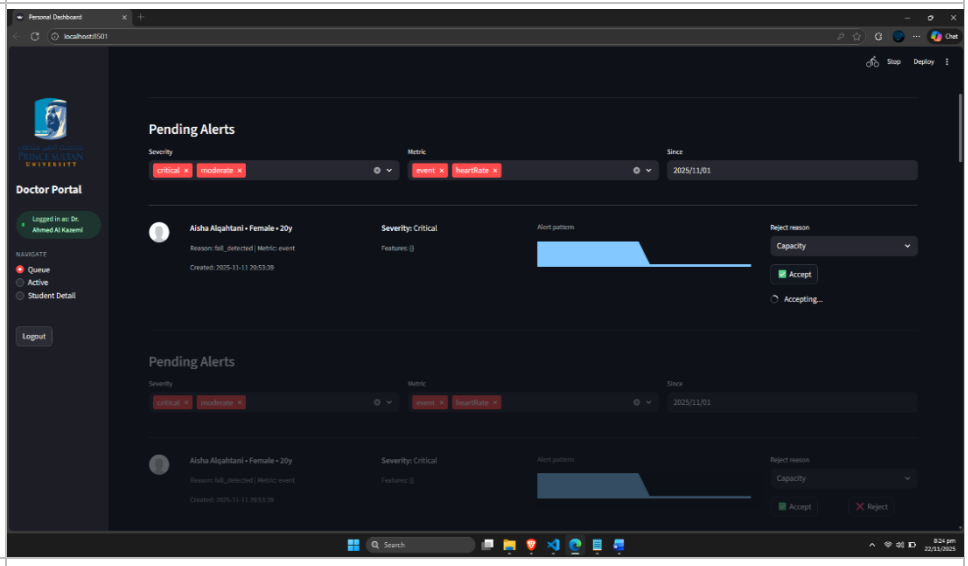
Test Case: UTC-FE-S3-02

Table 28: UTC-FE-S3-02

Test Case ID	UTC-FE-S3-02
Requirement ID	FE-S3-02
Preconditions	Alert exists in queue
Steps	1) Click alert. 2) Open detail view.
Expected Result	Alert detail shows vitals trend and reasoning.
Actual Result	
Pass/Fail	Pass

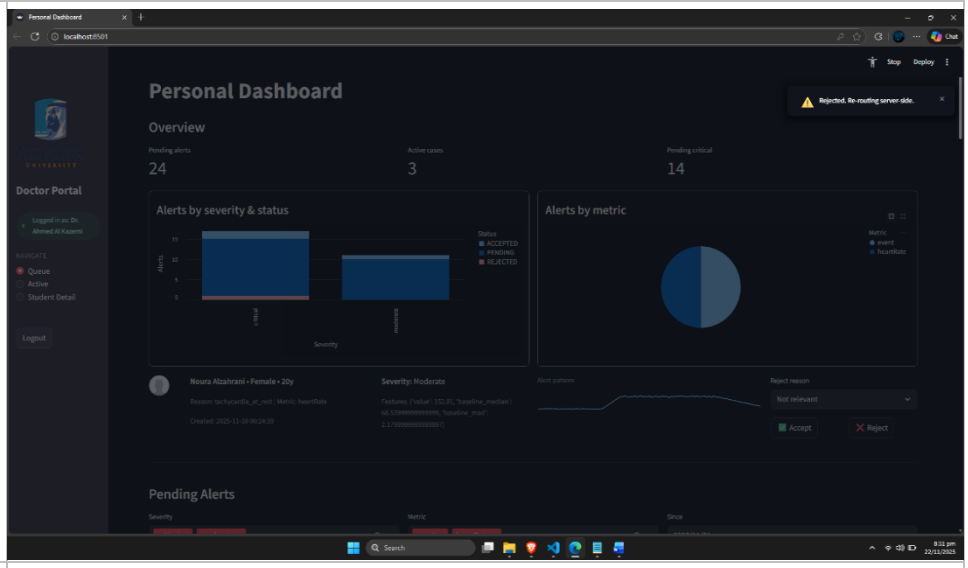
Test Case: UTC-FE-S3-03

Table 29: UTC-FE-S3-03

Test Case ID	**UTC-FE-S3-03**
Requirement ID	FE-S3-03
Preconditions	Alert is routed
Steps	1) Open alert. 2) Check Accept button.
Expected Result	Accept button visible and clickable.
Actual Result	
Pass/Fail	Pass

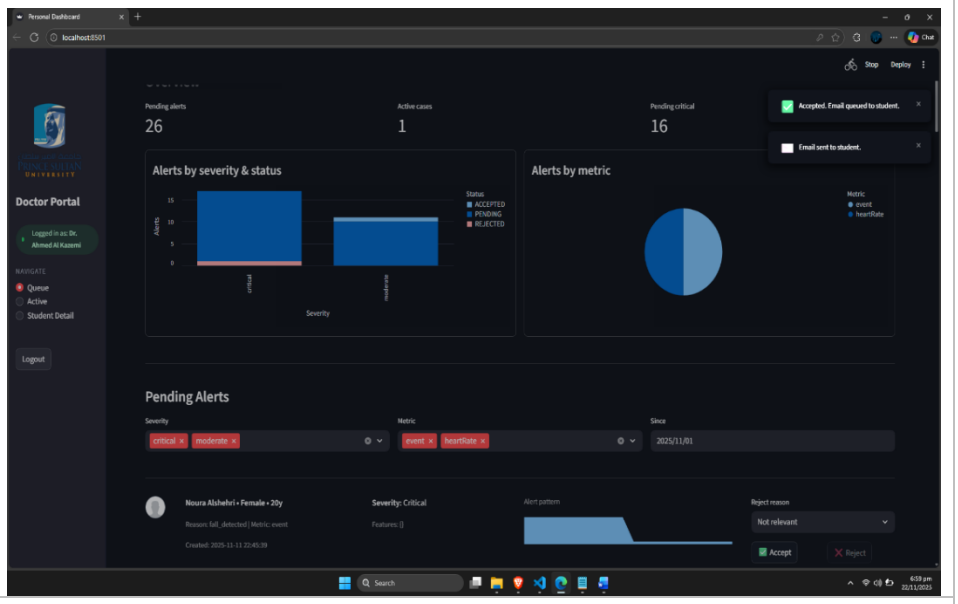
Test Case: UTC-FE-S3-04

Table 30: UTC-FE-S3-04

Test Case ID	**UTC-FE-S3-04**
Requirement ID	FE-S3-04
Preconditions	Alert is routed
Steps	1) Open alert. 2) Check Reject button.
Expected Result	Reject button visible and clickable.
Actual Result	
Pass/Fail	Pass

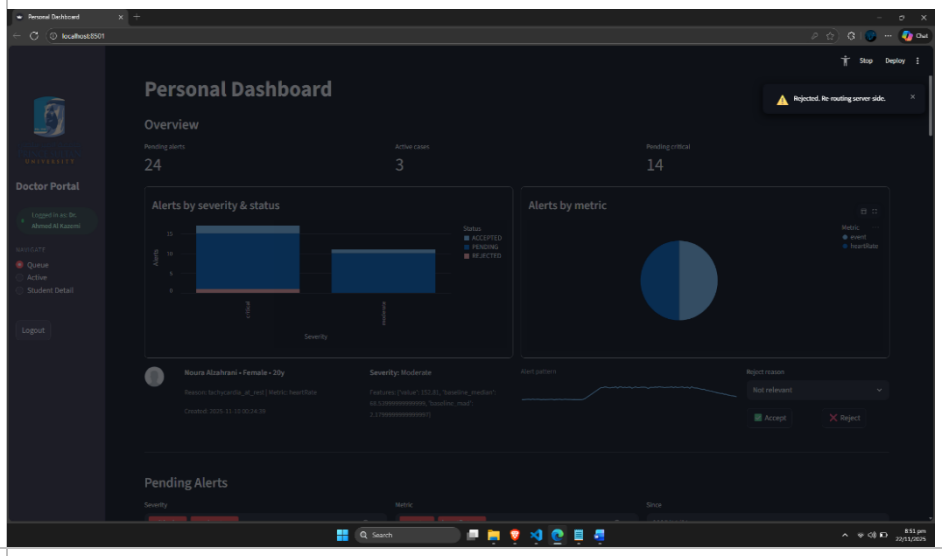
Test Case: UTC-FE-S3-05

Table 31: UTC-FE-S3-05

Test Case ID	**UTC-FE-S3-05**
Requirement ID	FE-S3-05
Preconditions	Alert detail open
Steps	1) Press Accept. 2) Wait for UI update.
Expected Result	Status becomes Accepted and success message appears.
Actual Result	
Pass/Fail	Pass

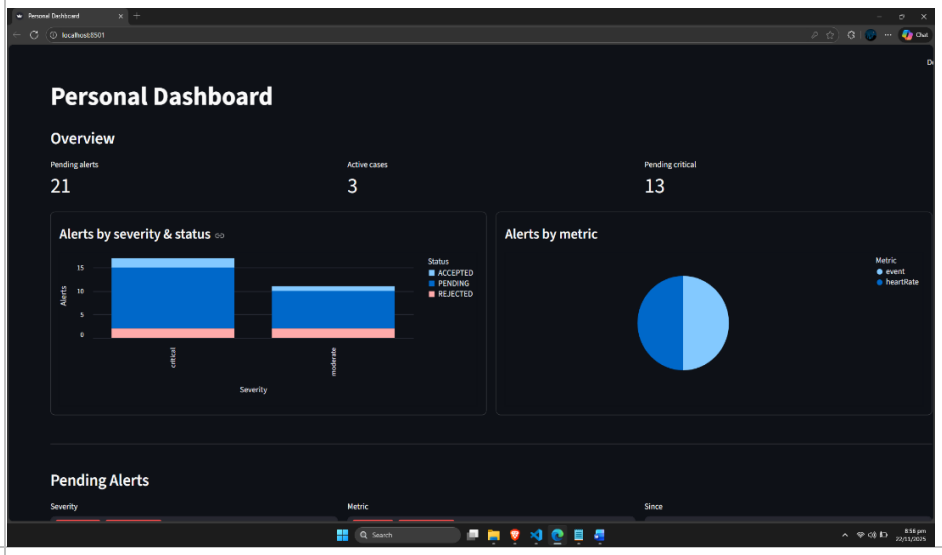
Test Case: UTC-FE-S3-06

Table 32: UTC-FE-S3-06

Test Case ID	**UTC-FE-S3-06**
Requirement ID	FE-S3-06
Preconditions	Alert detail open
Steps	1) Press Reject. 2) Wait for UI update.
Expected Result	Status becomes Rejected with confirmation message.
Actual Result	
Pass/Fail	Pass

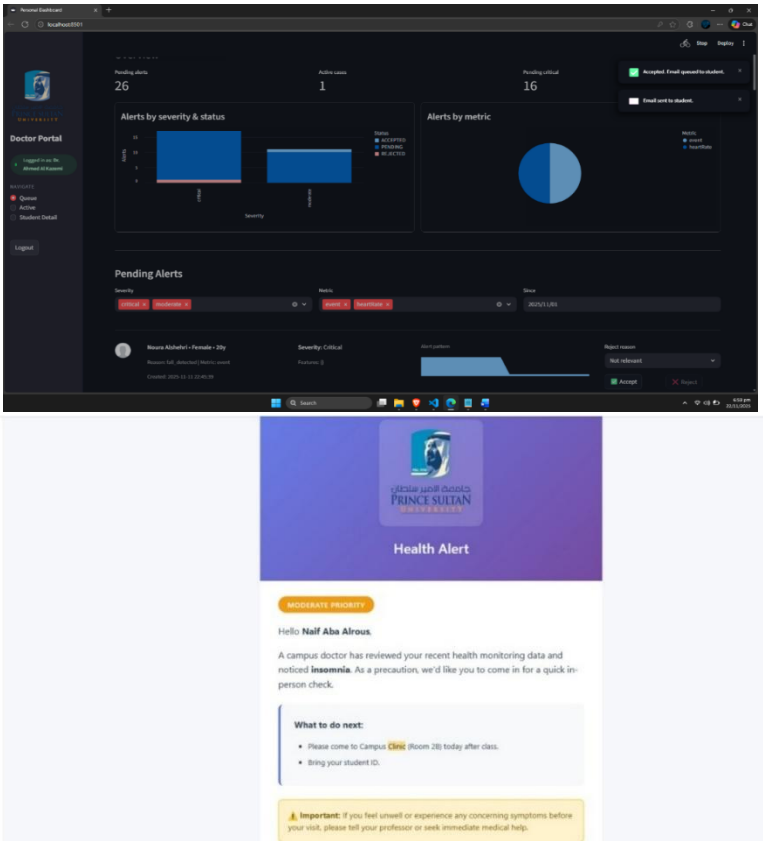
Test Case: UTC-FE-S3-07

Table 33:UTC-FE-S3-07

Test Case ID	**UTC-FE-S3-07**
Requirement ID	FE-S3-07
Preconditions	Alert updated previously
Steps	1) Refresh dashboard. 2) View list.
Expected Result	Updated alert status shown.
Actual Result	
Pass/Fail	Pass

Test Case: UTC-FE-S3-08

Table 34: UTC-FE-S3-08

Test Case ID	UTC-FE-S3-08
Requirement ID	FE-S3-08
Preconditions	Email simulated
Steps	1) Accept alert. 2) System sends email.
Expected Result	Email is sent to the user
Actual Result	
Pass/Fail	Pass

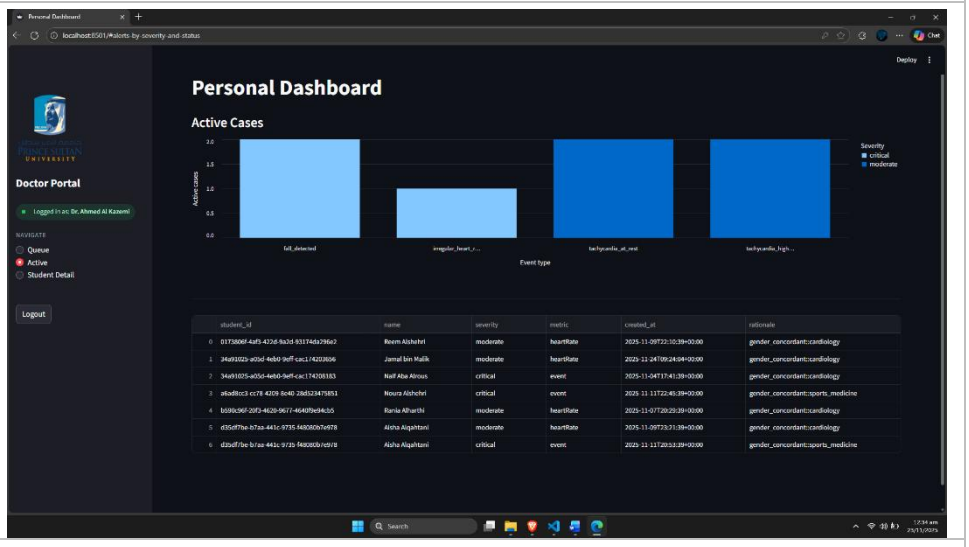
Test Case: UTC-FE-S3-09

Table 35: UTC-FE-S3-09

Test Case ID	**UTC-FE-S3-09**
Requirement ID	FE-S3-09
Preconditions	Alert has readings
Steps	1) Open alert. 2) Check graph.
Expected Result	Trend graph loads correctly.
Actual Result	
Pass/Fail	Pass

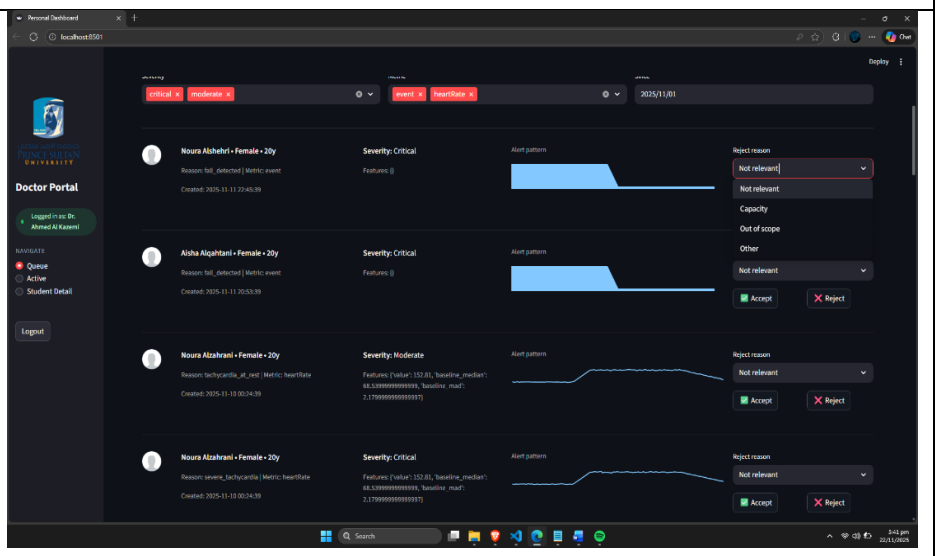
Test Case: UTC-FE-S3-10

Table 36: UTC-FE-S3-10

Test Case ID	UTC-FE-S3-10
Requirement ID	FE-S3-10
Preconditions	Alert has reasoning
Steps	1) Open alert. 2) View reasoning text.
Expected Result	Reasoning text displayed clearly.
Actual Result	
Pass/Fail	Fail

System Test Case: UTC-WF-02

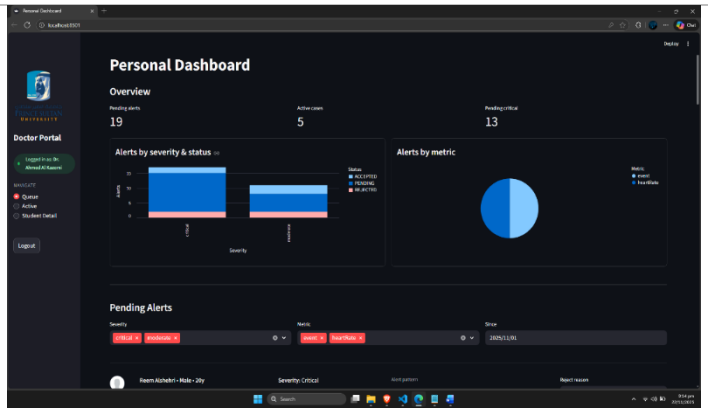
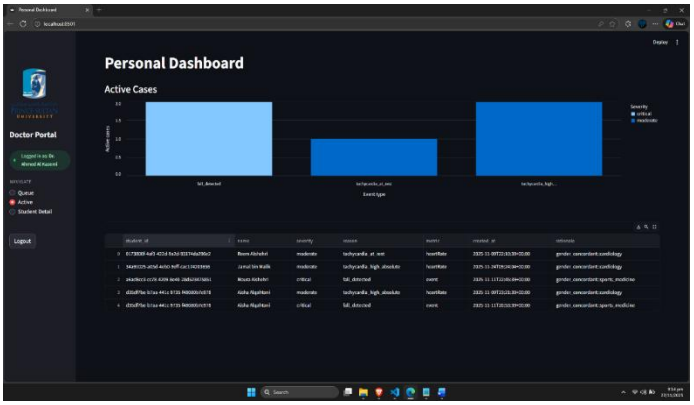
Table 37: UTC-WF-02

Test Case ID	UTC-WF-02
Requirement ID	FR-WF-02
Preconditions	Alert Generated
Steps	1.Try to change rejection reason
Expected Result	Doctor can change the rejection reason.
Actual Result	
Pass/Fail	Pass

Integration Test Cases

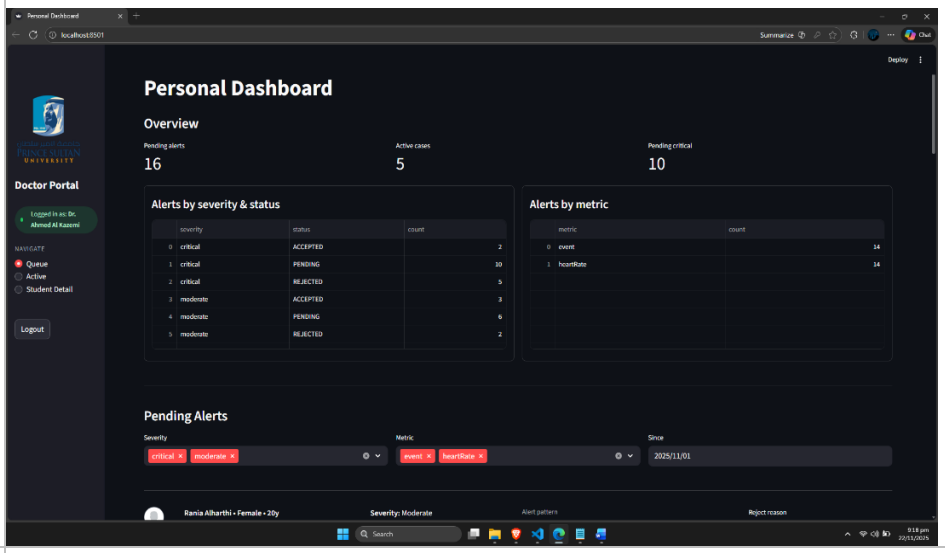
Test Case: ITC-FE-S3-01

Table 38: ITC-FE-S3-01

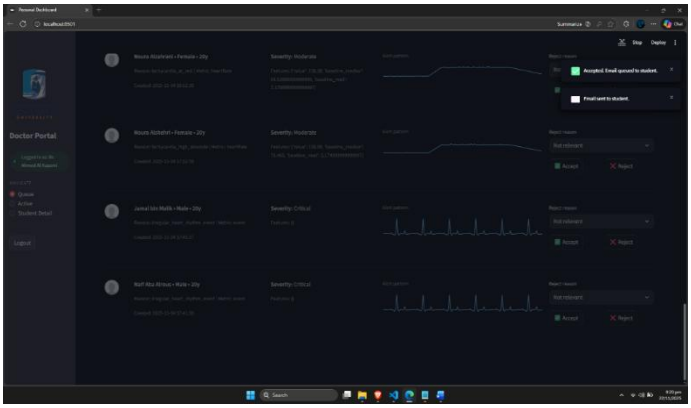
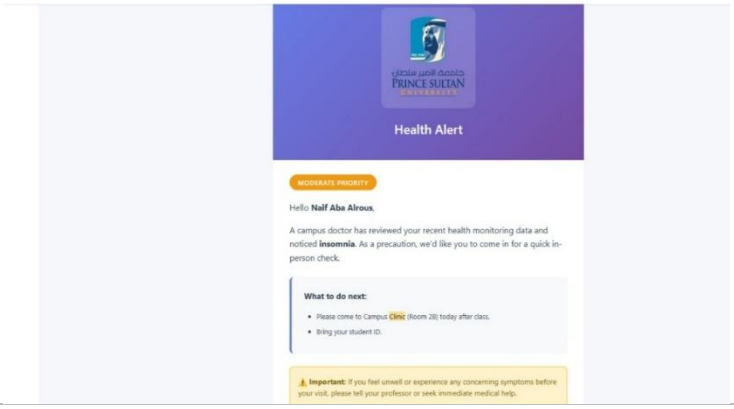
Test Case ID	**ITC-FE-S3-01**
Requirement ID	FE-S3-E2E-01
Preconditions	Alert routed
Steps	1) Open alert. 2) Accept. 3) Refresh dashboard.
Expected Result	Alert appears as Accepted across UI.
Actual Result	 
Pass/Fail	Pass

Test Case: ITC-FE-S3-02

Table 39: ITC-FE-S3-02

Test Case ID	**ITC-FE-S3-02**
Requirement ID	FE-S3-E2E-02
Preconditions	Alert routed
Steps	1) Open alert. 2) Reject. 3) Check dashboard.
Expected Result	Alert status Rejected and removed from queue.
Actual Result	
Pass/Fail	Pass

Test Case: ITC-FE-S3-03

Test Case ID	**ITC-FE-S3-03**
Requirement ID	FE-S3-E2E-03
Preconditions	Email system enabled
Steps	1) Accept alert. 2) UI shows success. 3) Email logged.
Expected Result	UI confirms acceptance and student notified.
Actual Result	 
Pass/Fail	Pass

Sprint-03 Review and Retrospective

What Went Well

- Gemini templates produced consistent and clear email communication.
- Monitoring features provided visibility into backend performance.
- Retry logic significantly improved email delivery reliability.

What Could Be Improved

- Email formatting could include adaptive phrasing for different alert severities.
- The monitoring dashboard still needs visualization improvements.

What We Will Try Next

- Enhancing email personalization details.
- Building a monitoring UI in Sprint 4.
- Running full workflow readiness tests for the pilot.

Sprint-03 Velocity

- Stories completed: 3
- Duration: 2 weeks
- Velocity trending stable compared to earlier sprints.

Sprint-04 Overview

Sprint 4 focused on validating the complete workflow for readiness before the pilot deployment. This sprint ensured that all components operated consistently when used together. The team ran simulations of real class sessions, verified all system KPIs, and finalized documentation and operational procedures.

Sprint-01 Goal

Confirm that the entire system is ready for pilot launch by completing full workflow simulations and validating accuracy, stability, and communication reliability.

User Stories Implemented

- As a stakeholder, I want to evaluate the system in a realistic scenario so that I can confirm it is safe and ready for pilot use.
- As a doctor, I want the system to behave consistently during simulated class sessions so that alerts and communication remain dependable.
- As an operator, I want pilot readiness checks completed so that the system can move to the deployment stage without major risks.

Tasks Implemented

- Conducted end-to-end workflow simulations for full alert evaluation and doctor decision paths.
- Validated all KPIs for performance, uptime, alert accuracy, and email delivery.
- Performed stability testing under simulated class load.
- Reviewed and corrected any issues in acceptance of email communication.
- Completed system documentation, including operational runbooks and support procedures.
- Verified backup, monitoring, and recovery steps.
- Completed readiness checklist for pilot approval.

Functional Requirements

Workflow Validation

- System shall support end-to-end workflow execution without interruption.
- System shall ensure that the doctor's decisions correctly trigger communication actions.
- System shall display alerts with correct reasoning and supporting details.

Reliability and Performance

- System shall meet defined KPI thresholds, including delivery speed, alert accuracy, and uptime.
- System shall maintain stable performance during load simulations.

Pilot Readiness

- System shall provide complete documentation for pilot staff.
- System shall ensure monitoring and logging are active and verifiable.
- System shall prevent deployment until no critical issues remain.

Class Diagram: Sprint 4

Class diagram for Sprint 4 illustrating pilot readiness components and their relationships, including simulation of class sessions, KPI calculation, load testing, readiness checklist evaluation, deployment management, monitoring dashboards, failure logging, and operational runbooks integrated with the existing alert and email workflow.

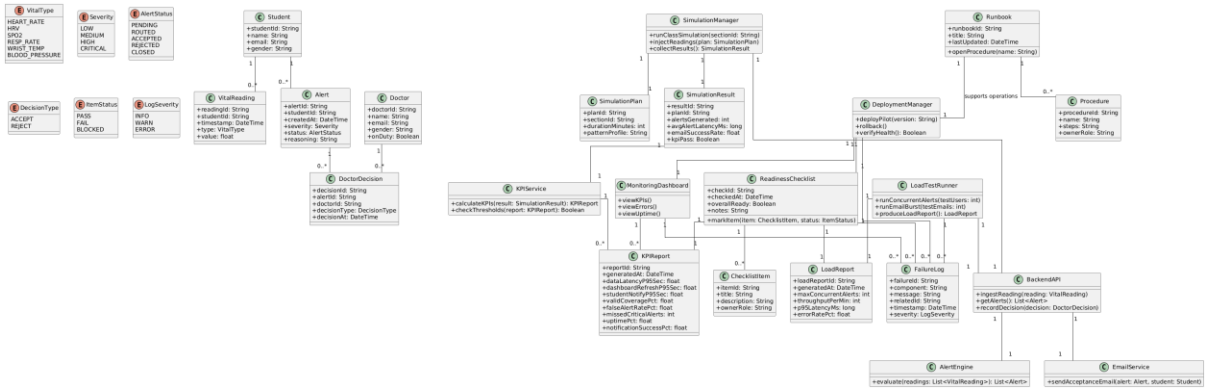


Figure 16: Class Diagram: Sprint 4

Activity Diagrams of Sprint 4

Flow 1 Flow showing how simulated class sessions generate reading patterns, trigger alerts, and produce a complete simulation result for pilot readiness evaluation.

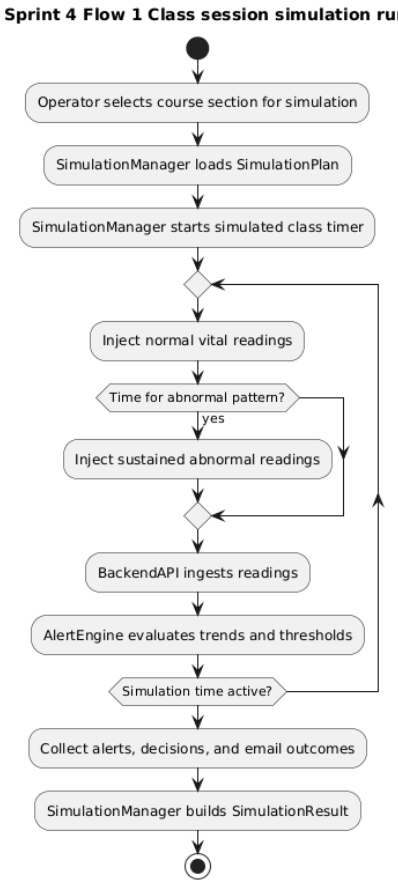


Figure 17: Activity Diagrams of Sprint 4-Flow 1

Flow 2 Flow showing the calculation of KPIs from simulation results, including latency, accuracy, coverage, uptime, and notification success, followed by validation against readiness thresholds.

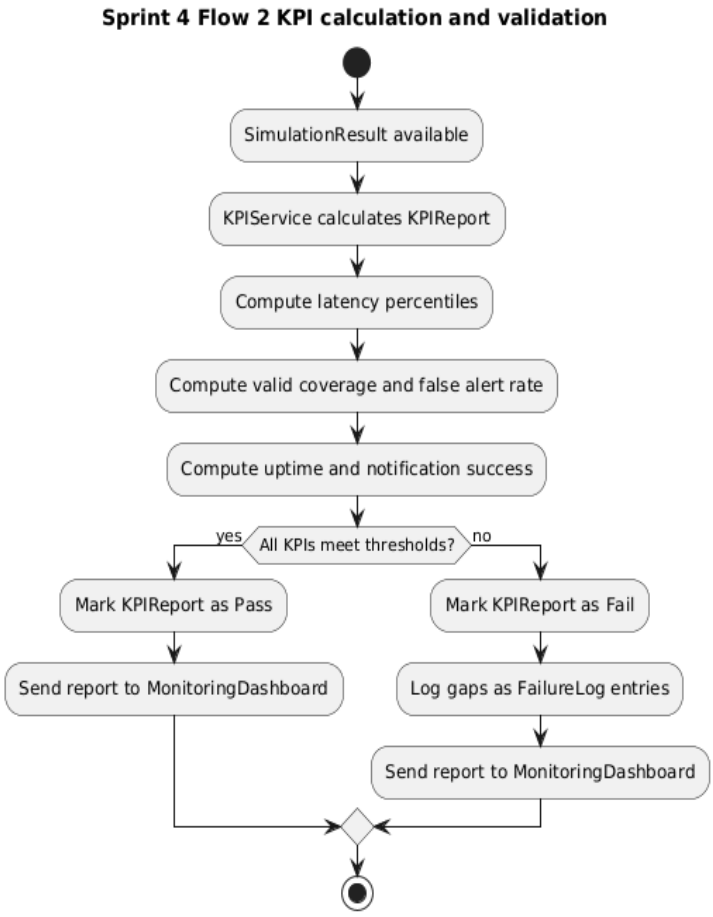


Figure 18: Activity Diagrams of Sprint 4-Flow 2

Flow 3 Flow showing load and stability testing where concurrent alerts and email bursts are executed, performance is measured, and a load report is generated with any failures logged.

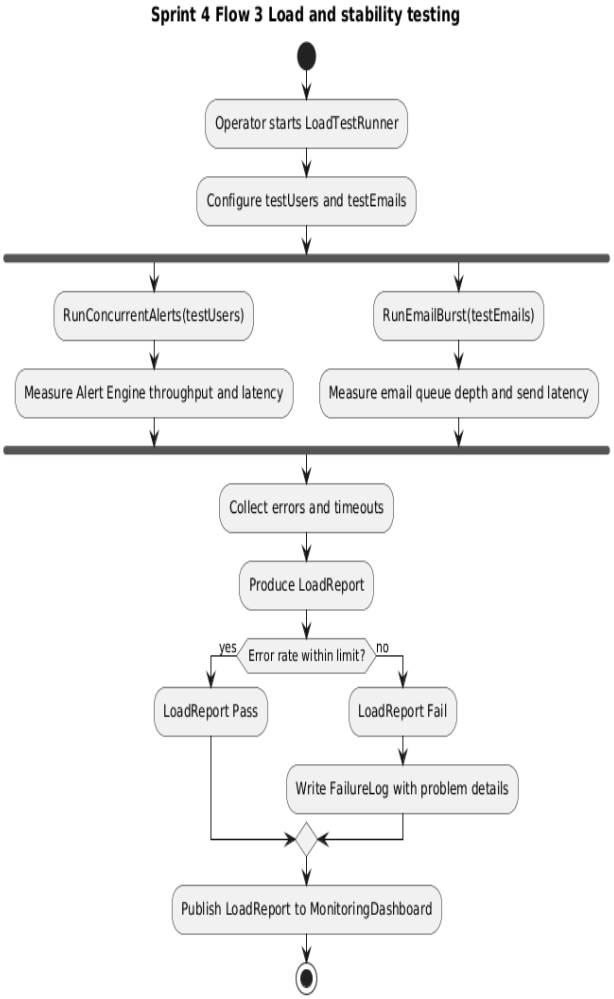


Figure 19: Activity Diagrams of Sprint 4-Flow 3

Flow 4 Flow showing the evaluation of the readiness checklist, combining KPI and load results with documentation reviews to determine whether the system is ready for the pilot.

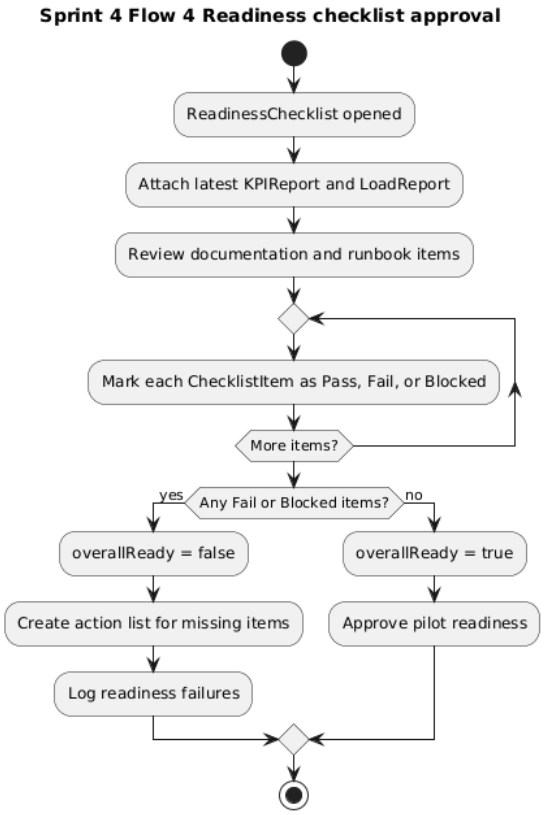


Figure 20: Activity Diagrams of Sprint 4-Flow 4

Flow 5 Flow showing the deployment process for the pilot, including readiness verification, deployment execution, post-deployment health checks, and rollback procedures if needed.

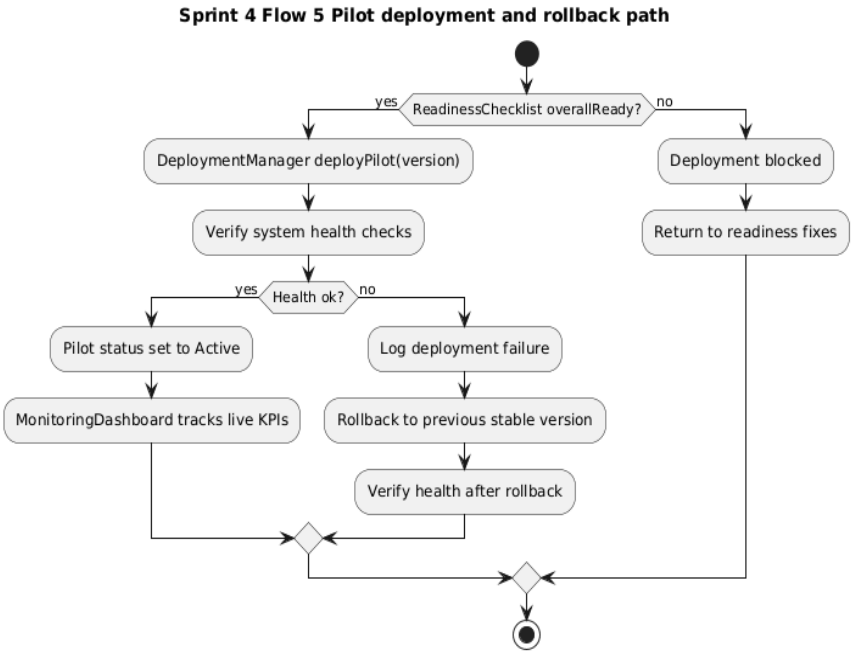
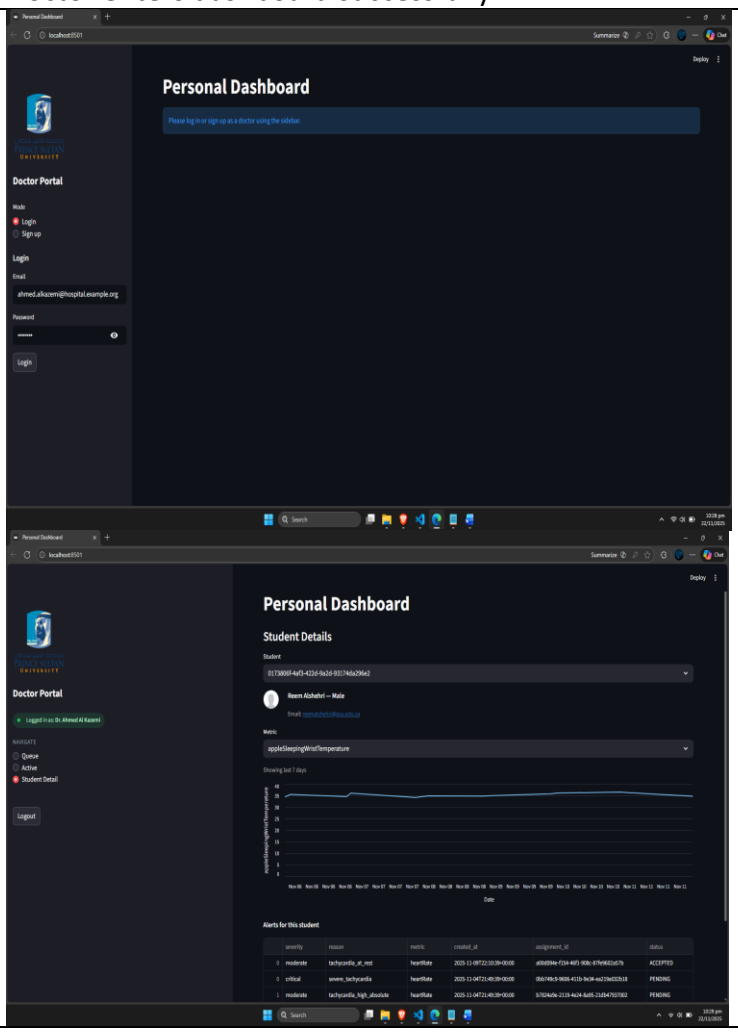


Figure 21: Activity Diagrams of Sprint 4-Flow 5

Unit Test Cases

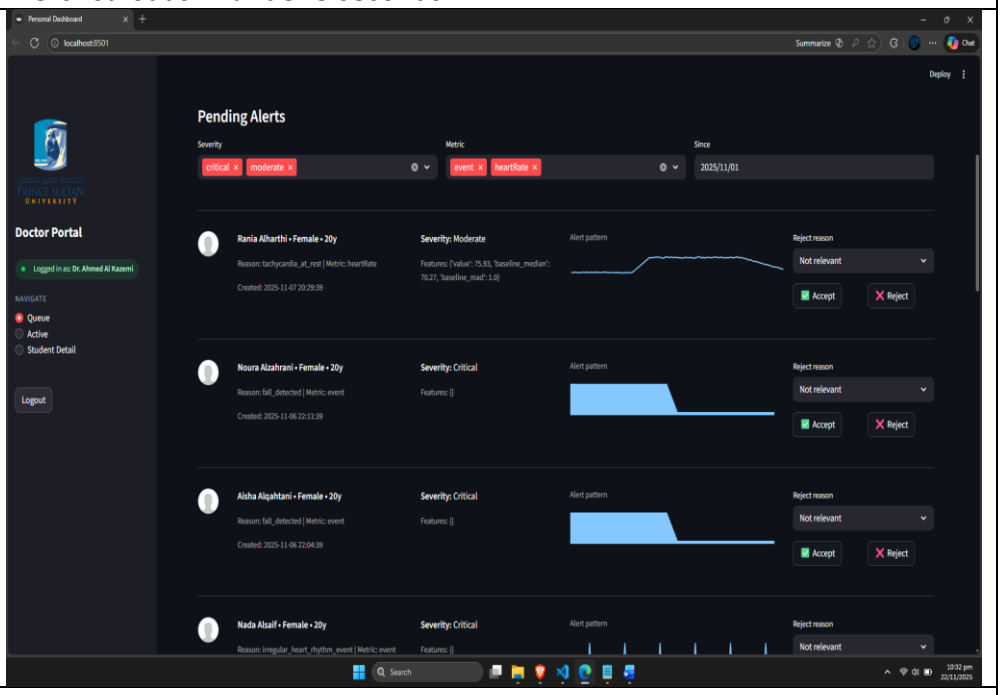
Test Case: UTC-FE-S4-01

Table 40: UTC-FE-S4-01

Test Case ID	**UTC-FE-S4-01**
Requirement ID	CHARTER-RBAC-01
Preconditions	Doctor account exists.
Steps	1) Open portal. 2) Enter credentials. 3) Login.
Expected Result	Doctor enters dashboard successfully.
Actual Result	
Pass/Fail	Pass

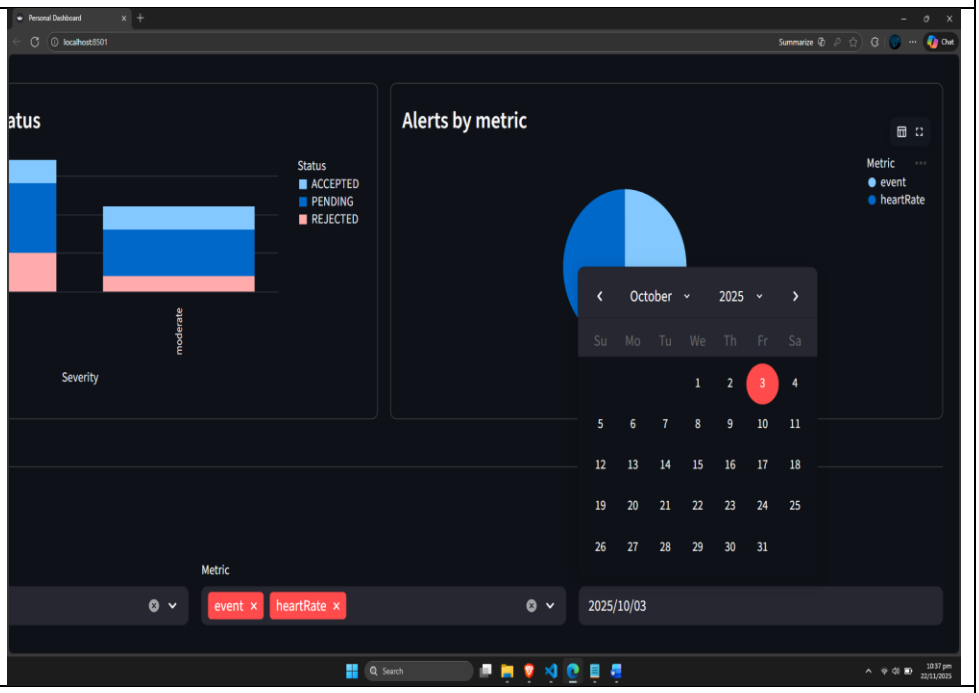
Test Case: UTC-FE-S4-02

Table 41: UTC-FE-S4-02

Test Case ID	**UTC-FE-S4-02**
Requirement ID	CHARTER-KPI-UI-01
Preconditions	Doctor logged in, alerts exist.
Steps	1) Refresh dashboard. 2) Measure load speed.
Expected Result	Alert list loads in under 3 seconds.
Actual Result	
Pass/Fail	Pass

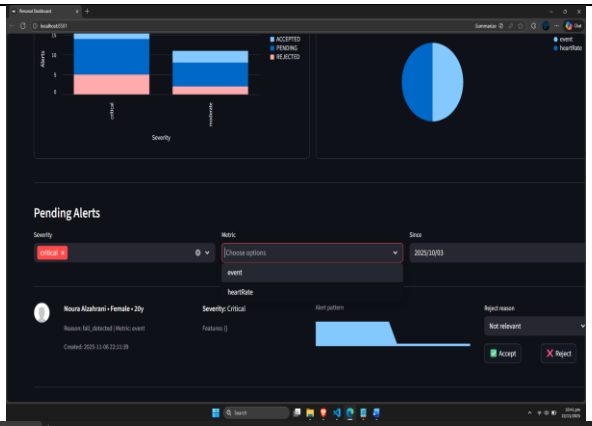
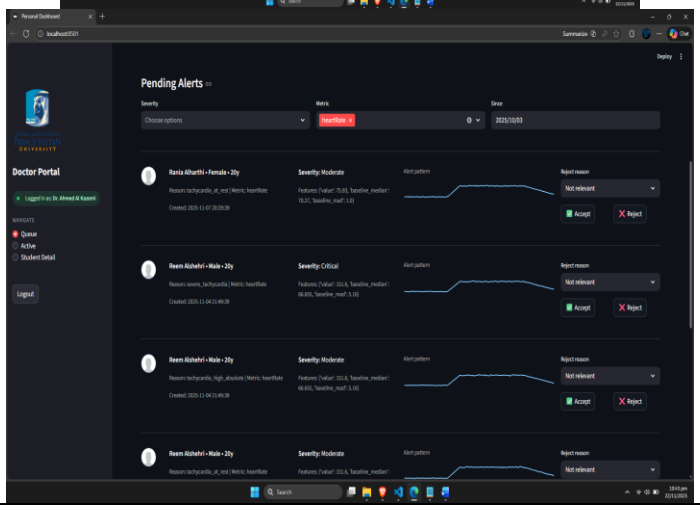
Test Case: UTC-FE-S4-03

Table 42: UTC-FE-S4-03

Test Case ID	**UTC-FE-S4-03**
Requirement ID	CHARTER-PRIV-01
Preconditions	Doctor logged in.
Steps	1.Open alerts page. 2.Open Date Filter 3.Set Date
Expected Result	Date Filter works
Actual Result	
Pass/Fail	Pass

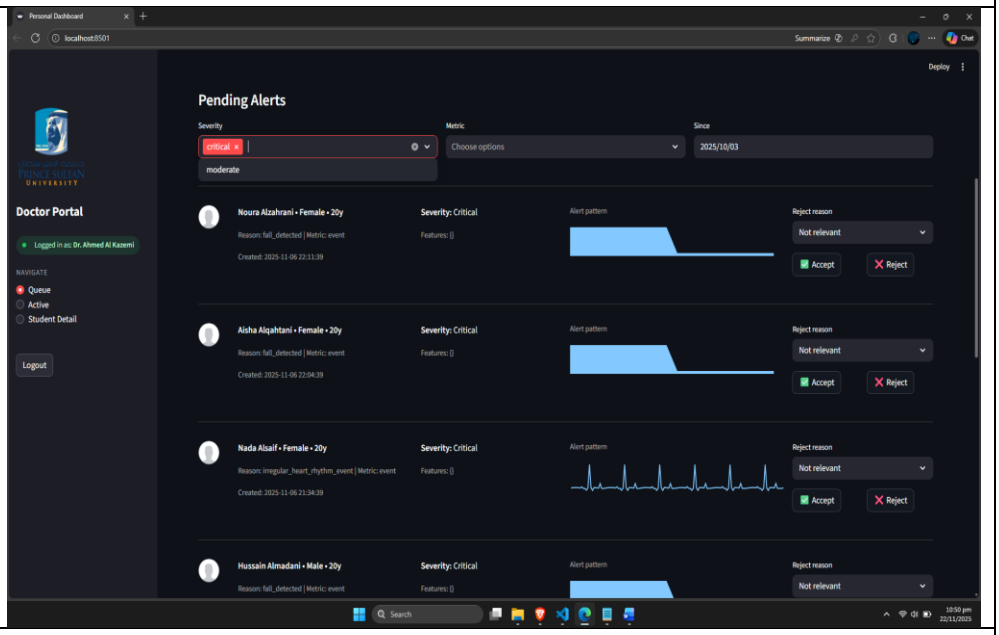
Test Case: UTC-FE-S4-04

Table 43: UTC-FE-S4-04

Test Case ID	**UTC-FE-S4-04**
Requirement ID	CHARTER-TRIAGE-01
Preconditions	Alert exists.
Steps	1.Open alerts page. 2.Set Metric Filter
Expected Result	Result is filtered
Actual Result	 
Pass/Fail	Pass

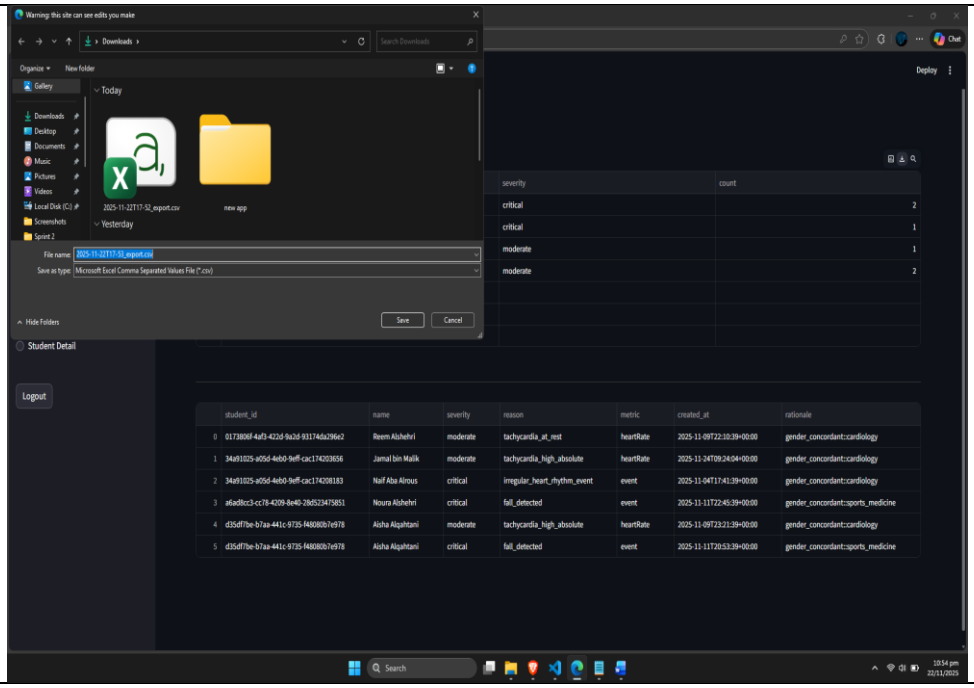
Test Case: UTC-FE-S4-05

Table 44: UTC-FE-S4-05

Test Case ID	**UTC-FE-S4-05**
Requirement ID	CHARTER-DECISION-01
Preconditions	Alert exists.
Steps	1)Open alerts page. 2)Set Severity Filter
Expected Result	Result is filtered
Actual Result	
Pass/Fail	Pass

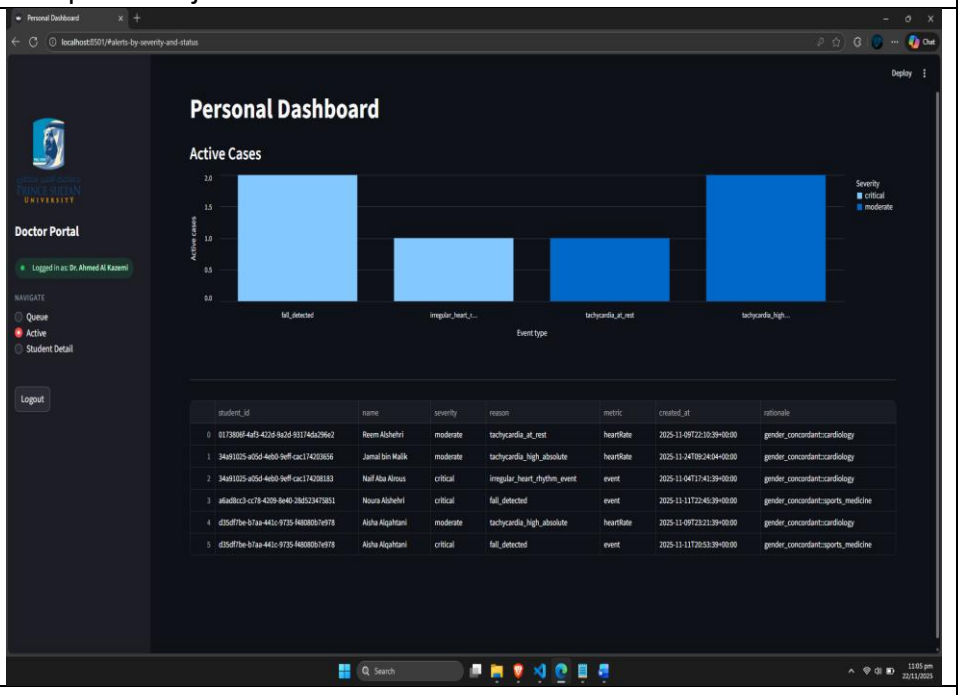
Test Case: UTC-FE-S4-06

Table 45: UTC-FE-S4-06

Test Case ID	**UTC-FE-S4-06**
Requirement ID	CHARTER-DECISION-02
Preconditions	Alert in routed state.
Steps	1) Open Active alert 2) Export Data
Expected Result	Data is Downloaded
Actual Result	
Pass/Fail	Pass

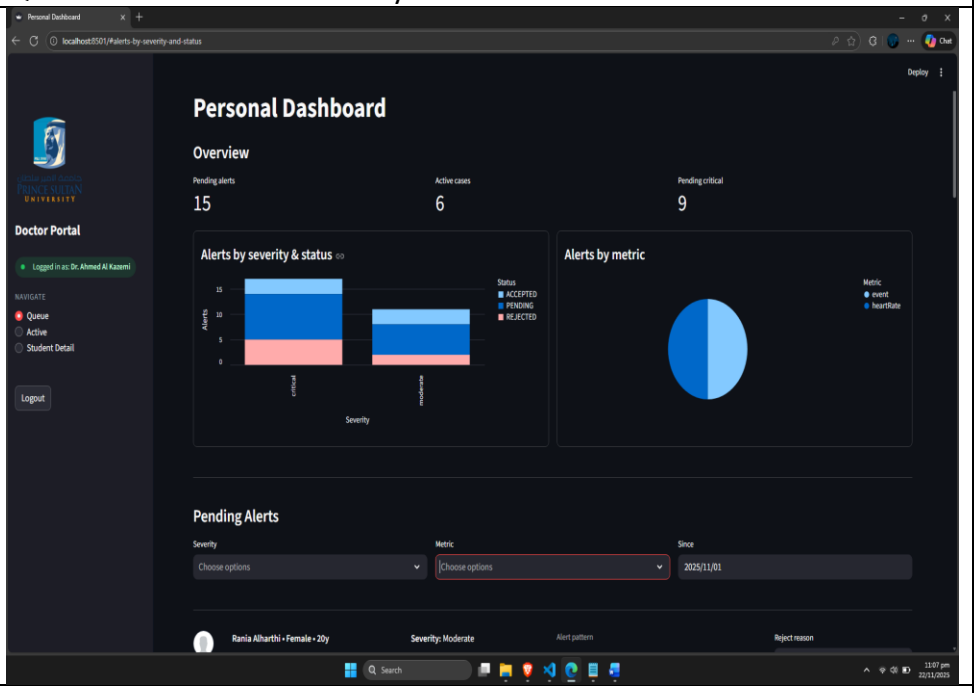
Test Case: UTC-FE-S4-07

Table 46: UTC-FE-S4-07

Test Case ID	**UTC-FE-S4-07**
Requirement ID	CHARTER-WF-01
Preconditions	Alert already accepted.
Steps	1) Open alert.
Expected Result	Accept and Reject buttons hidden.
Actual Result	
Pass/Fail	Pass

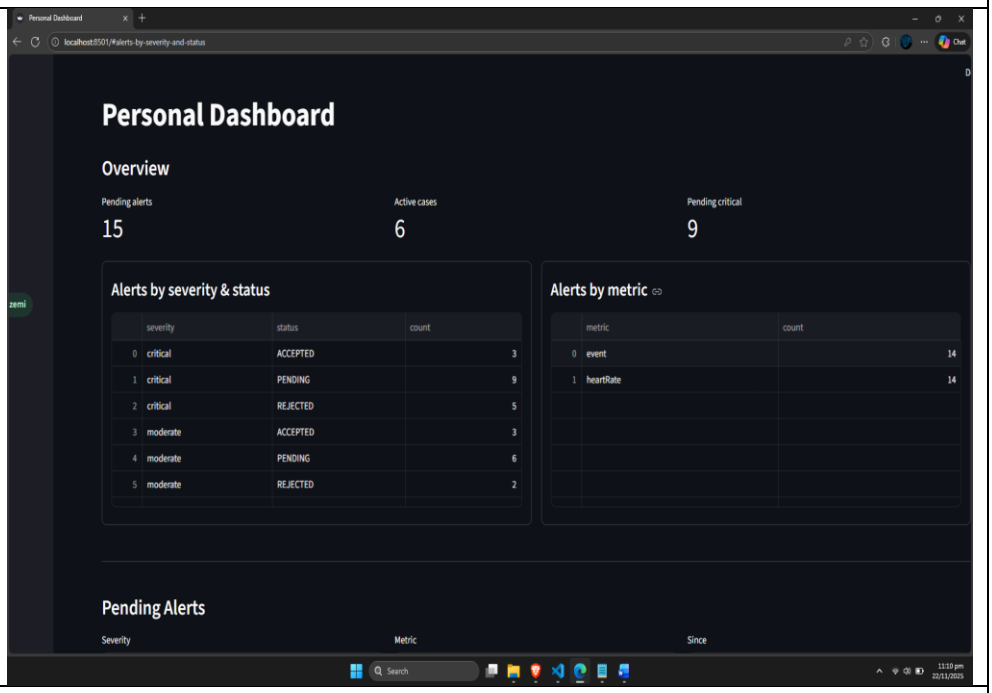
Test Case: UTC-FE-S4-08

Table 47: UTC-FE-S4-08

Test Case ID	**UTC-FE-S4-08**
Requirement ID	CHARTER-KPI-UI-02
Preconditions	Dashboard open.
Steps	1) Leave dashboard open. 2) Wait refresh cycle.
Expected Result	Queue refreshes automatically.
Actual Result	
Pass/Fail	Pass (Refreshes after every 20 seconds)

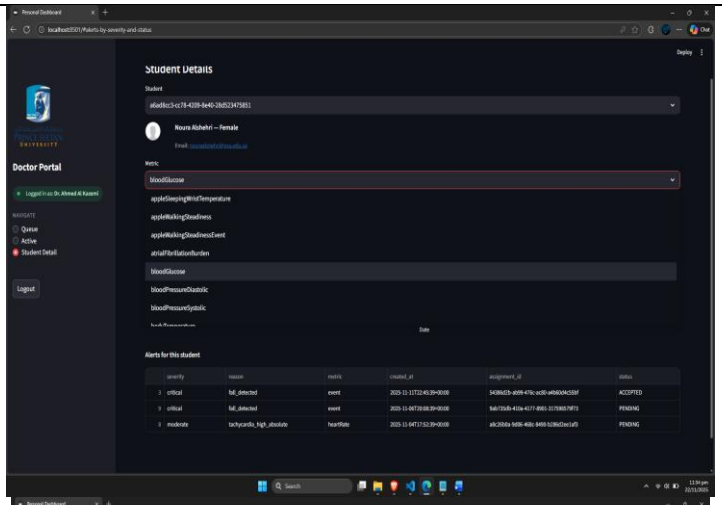
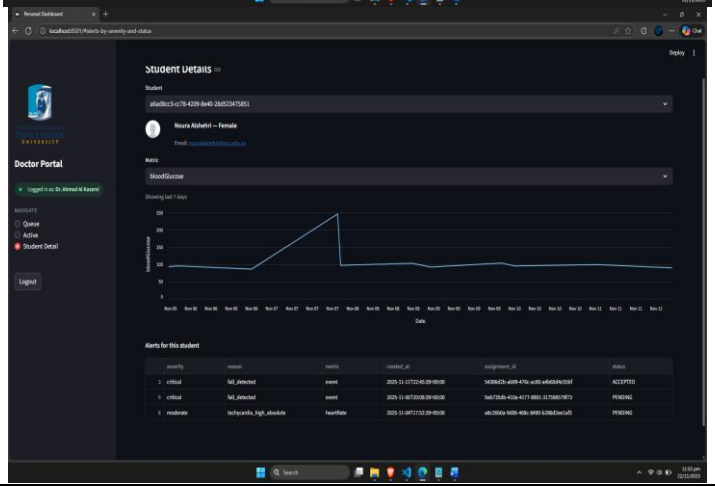
Test Case: UTC-FE-S4-09

Table 48: UTC-FE-S4-09

Test Case ID	**UTC-FE-S4-09**
Requirement ID	CHARTER-PILOT-01
Preconditions	KPI report exists.
Steps	1) Open the pilot readiness page.
Expected Result	Shows KPI summary clearly.
Actual Result	
Pass/Fail	Pass

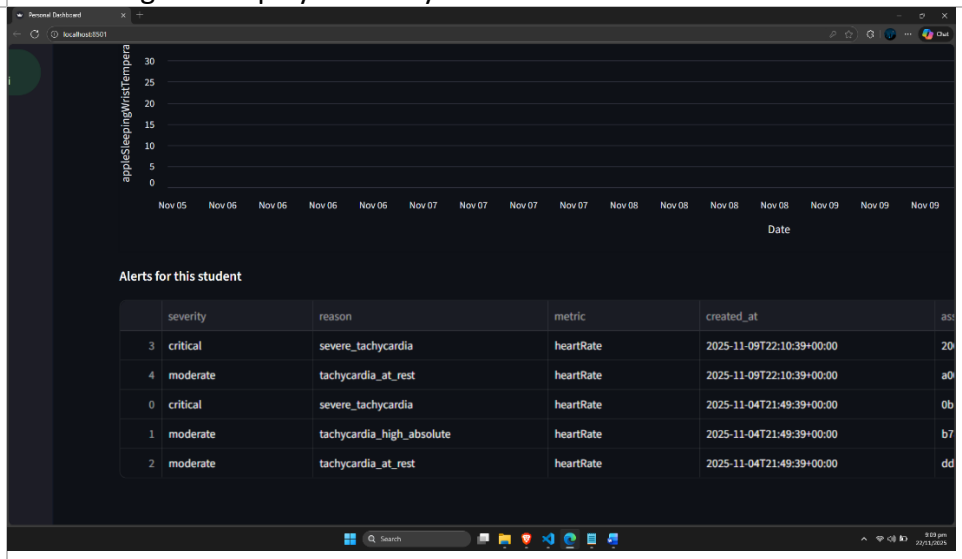
Test Case: UTC-FE-S4-10

Table 49: UTC-FE-S4-10

Test Case ID	**UTC-FE-S4-10**
Requirement ID	CHARTER-REL-01
Preconditions	Alerts Generated
Steps	1)Select Student 2)Select Metric
Expected Result	Various Metrics are recorded and Graph updates
Actual Result	 
Pass/Fail	Pass

System Test Case: UTC-FE-S3-10

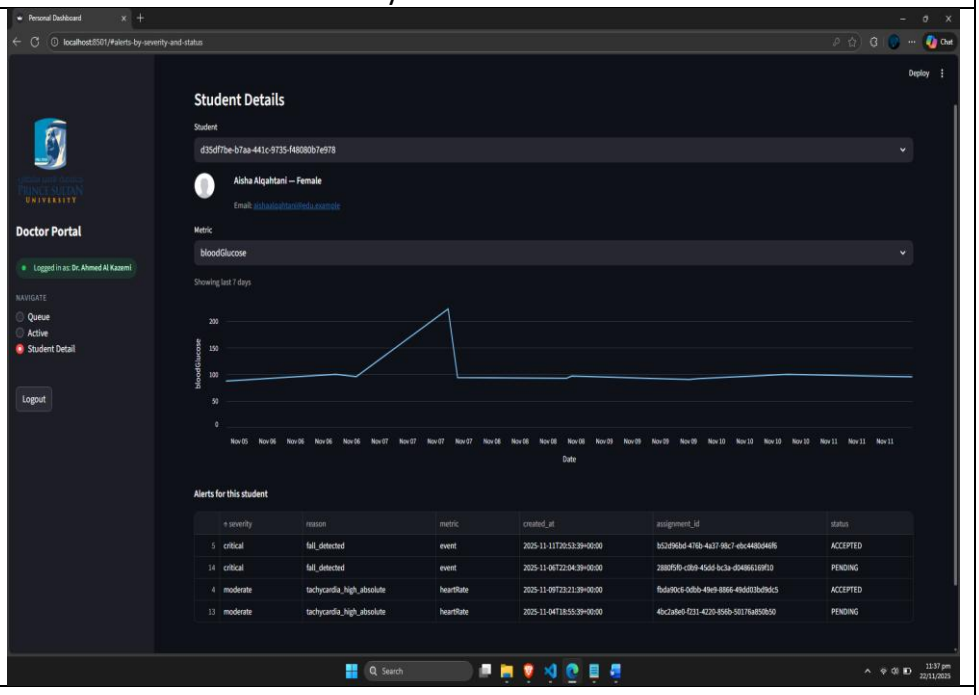
Table 50: UTC-FE-S3-10

Test Case ID	**UTC-FE-S3-10**
Requirement ID	FE-S3-10
Preconditions	Alert has reasoning
Steps	1) Open alert. 2) View reasoning text.
Expected Result	Reasoning text displayed clearly.
Actual Result	
Pass/Fail	Pass

Integration Test Cases

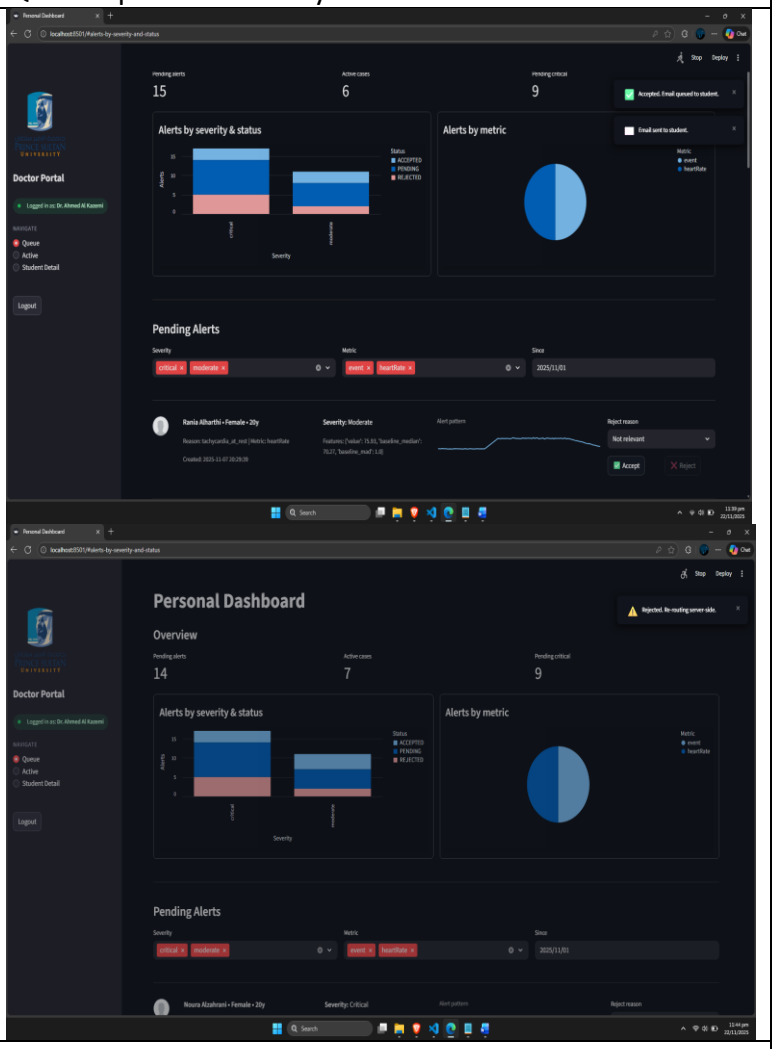
Test Case: ITC-FE-S4-01

Table 51: ITC-FE-S4-01

Test Case ID	**ITC-FE-S4-01**
Requirement ID	CHARTER-E2E-01
Preconditions	Alerts Generated
Steps	1) Login. 2) Student Detail 3) Select Filters 4) See Alerts
Expected Result	Alert shown based on severity
Actual Result	
Pass/Fail	Pass

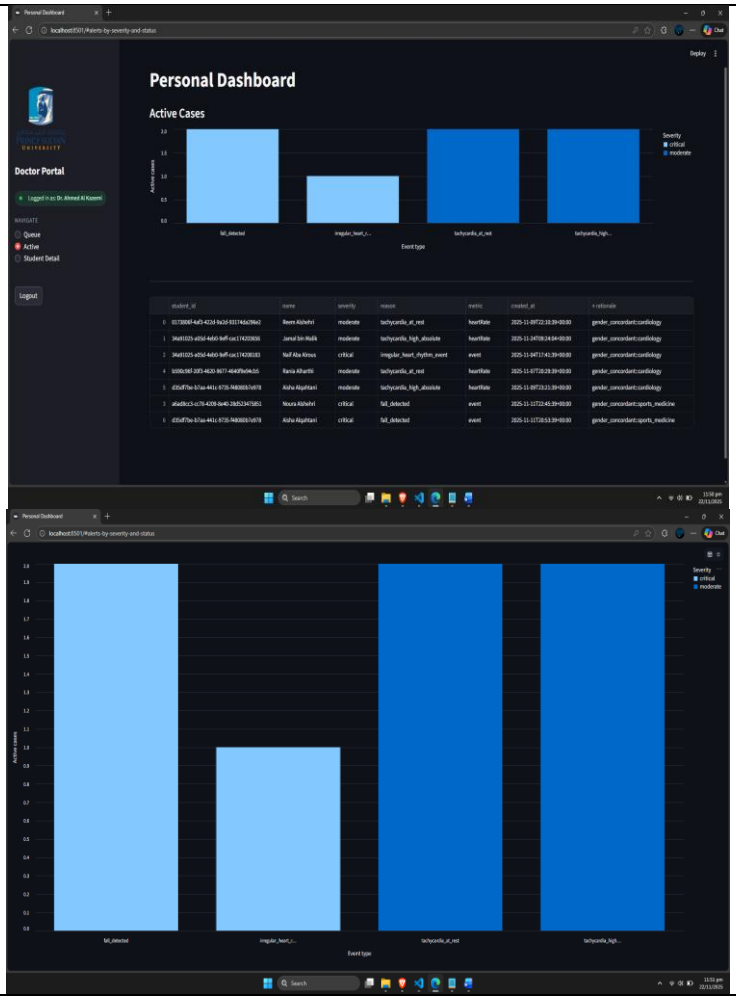
Test Case: ITC-FE-S4-02

Table 52: ITC-FE-S4-02

Test Case ID	**ITC-FE-S4-02**
Requirement ID	CHARTER-PILOT-02
Preconditions	Multiple alerts exist.
Steps	1) Open dashboard. 2) Accept alert A. 3) Reject alert B.
Expected Result	Queue updates correctly for both alerts.
Actual Result	
Pass/Fail	Pass

Test Case: ITC-FE-S4-03

Table 53: ITC-FE-S4-03

Test Case ID	**ITC-FE-S4-03**
Requirement ID	CHARTER-KPI-01
Preconditions	Alerts Generated
Steps	1)Open Active Cases page 2)View Summary 3)Sort Summary 4)Enlarge Graphs
Expected Result	Active Cases Page Works Efficiently
Actual Result	
Pass/Fail	Pass

Sprint-04 Review and Retrospective

What Went Well

- End-to-end workflow is consistent and stable.
- KPIs met or exceeded acceptable thresholds.
- Documentation completed and aligned with pilot needs.

What Could Be Improved

- More automation tools could assist in load simulation.
- The monitoring dashboard could be expanded for better visibility.

What We Will Try Next

- Prepare system for pilot deployment.
- Provide training to clinic staff.
- Schedule first monitored pilot session.

Sprint Velocity

- Stories completed: 3
- Duration: 2 weeks
- Velocity consistent and stable across the project.

Application usability testing

Usability Test Case (5 users)

Test Case ID: USTC-SMART-01

Title: Usability test of Smart+ doctor dashboard and alert workflow with 5 users

Objective: Confirm that clinic staff can triage alerts, review vitals, and trigger student communication in Smart+ without help, and that core screens are clear and responsive for the pilot workflow.

Smart+ Report

Participants: 5 members acting as simulated doctors to represent the planned pilot users, not licensed medical professionals.

Preconditions:

- Smart+ web portal available on clinic PCs.
- Test accounts created with realistic alert data in the queue.
- Users briefed on the classroom health monitoring goal, but not on detailed steps.

Tasks:

1. Sign in to Smart+ and reach the main alerts queue.
2. Open a pending alert, review vitals trends and reasoning text, and describe the situation in their own words.
3. Accept an alert and confirm that the decision is logged and the case moves out of the queue.
4. Reject a different alert and enter a clear rejection reason.
5. Use date and severity filters to focus on critical alerts from the last 24 hours.
6. Open the KPI dashboard and locate the current alert volume and acceptance rate.
7. Confirm that the queue refreshes automatically and that changes in status are reflected on screen.

Metrics:

- Task completion rate per step.
- Number and type of errors or requests for assistance.
- Time to complete full triage flow (from login to last task).
- Post task ratings of clarity, ease of use, and confidence in using Smart+ during real class sessions.

Results summary (5 participants)

- 5 users (100%) signed in and reached the alerts queue without help.
- 4 users (80%) found the alert detail view very clear, and 1 user (20%) found it somewhat clear.
- 5 users (100%) completed the accept and reject process. 3 users (60%) said it was very easy and 2 users (40%) said it was somewhat easy.
- Filters were helpful for most: 3 users (60%) rated them very useful, 1 user (20%) somewhat useful, and 1 user (20%) neutral.
- Automatic queue refresh was very helpful for 4 users (80%), while 1 user (20%) did not notice it.

- KPI dashboard understanding was strong: 3 users (60%) understood it immediately and 2 users (40%) needed a short explanation.

Graph of usability results

A bar chart can summarize for each task the percentage of users who rated it very easy, somewhat easy, or difficult, plus an overall task completion rate of 100 percent for the 5 pilot users.